



11

Дискретни структури

доц. д-р Кристина Близнакова
доц. д-р Владимир Николов

2021-2022 , ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

Съдържание

11.1. Представяне на моделите в паметта.

11.2. Задачи

11.3. Моделиране на паралелни процеси

11. 1 Как представяме моделите в паметта?

Пример: проблемът с вируса и здравите клетки?

Дефиниране на проблема

Средство за решаване: моделираме с мрежи на Петри.

Как да го представим в паметта? Чрез **абстрактни структури**

Абстрактни структури

- ☐ Абстрактните структури са средство за представяне на моделите в паметта.
- ☐ Те засягат само модела, концепцията.
- ☐ Не се засягат детайли по имплементацията.
- ☐ Стек и опашка са примери за абстрактни структури.
- ☐ Масивът НЕ е пример за абстрактна структура.

Разлика

- ❑ Стек & Опашка срещу Масив
- ❑ Стек – контейнер, който позволява push и pop
- ❑ Опашка – контейнер, при който елементи се добавят само най-отзад (**enqueue**) и се извличат само най-отпред (**dequeue**).
- ❑ Не ни интересуват детайлите по имплементацията.
- ❑ В масив, всеки елемент може да бъде достъпен, докато в абстрактните структури от данни достъпът е ограничен.

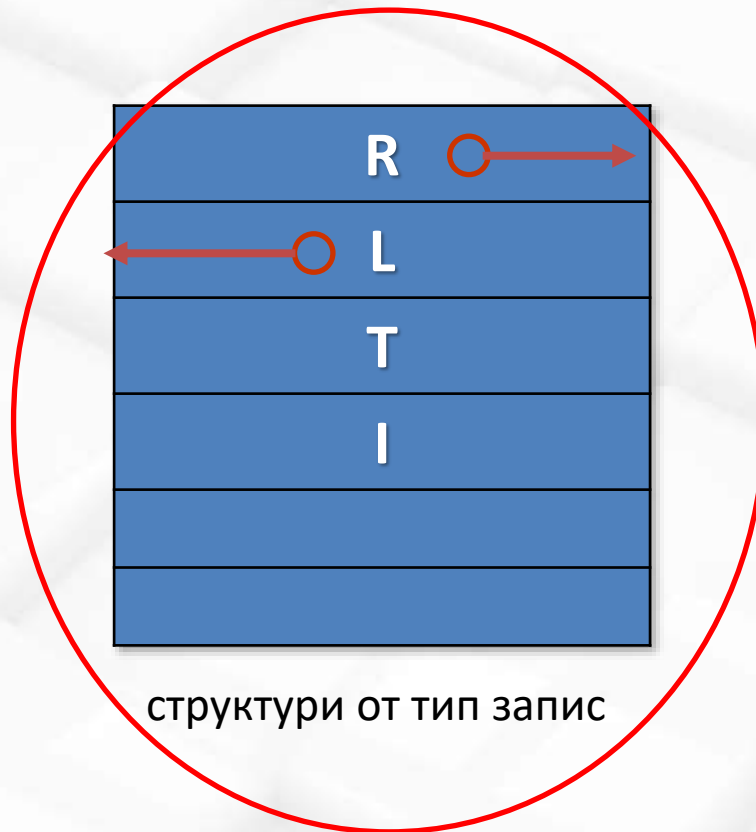
Видове абстрактни структури

- ☐ Стек
- ☐ Опашка
- ☐ Списък



Линейни (списъчни) структури
от данни

Пример:

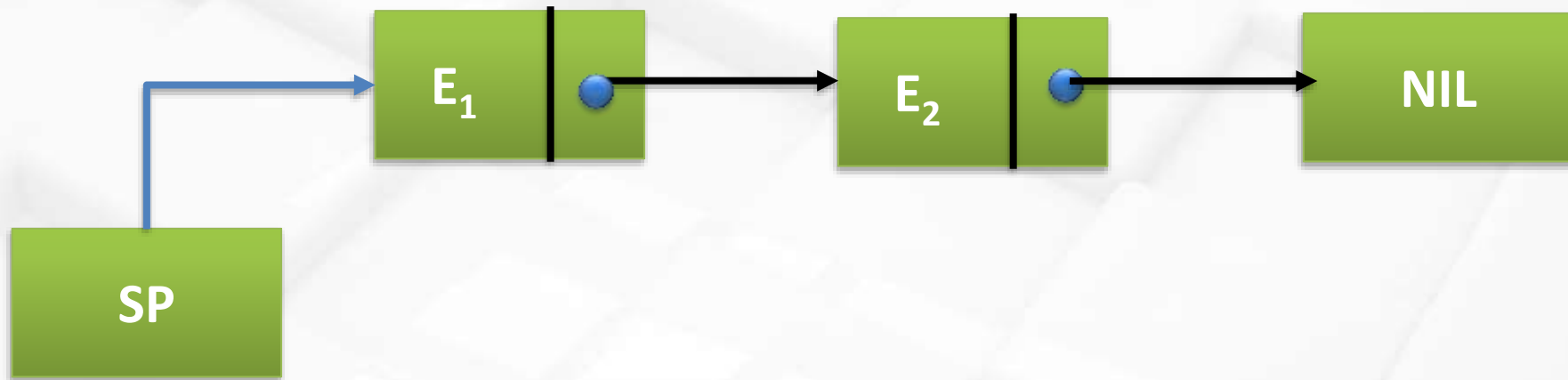


За представяне на информа-
ционни възли (място, преход)
се използват структури от тип
запис

A

Стек

- ❑ Позволява достъп само до последния въведен елемент.
- ❑ Елемент се вмъква или извлича (премахва) от стека от единия край наречен **връх на стека**.
- ❑ Този механизъм се нарича **LIFO** - Last-In-First-Out.



Стек

Пример за имплементация чрез собствен код(лекция 5 ООП):

```
template <class T>
struct ptr{T key; ptr *next;};
template <class T>
class CStackT {
ptr<T> *sp;
public:
CStackT() {sp = NULL;}
~CStackT();
void push(const T &key);
bool pop(T &result);
};
```

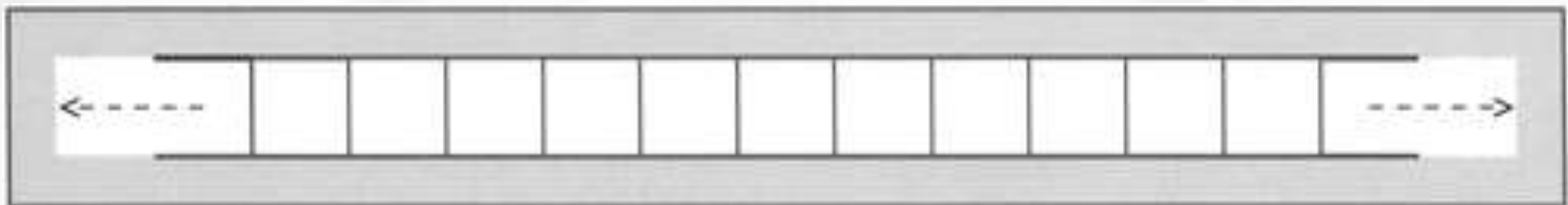
Стек

Пример за STL stack:

```
{
    stack<string, deque<string>> st; //създаване на стек от низове
    st.push("E2");  st.push("E1");    // добавяне на 2 елемента
    cout << st.top(); st.pop(); // четене и извеждане на два от елементите
    cout << st.top(); st.pop();
    st.pop(); // премахване на елемент без извеждане
    cout << "current stack size: " << st.size() << endl; // извеждане големината
    if (!st.empty()) { //Проверка за празен
        cout << st.top();
        st.pop();
    }
}
```

Опашка

- ❑ Опашка е абстрактна структура за данни, подобна на стек, освен че първият вмъкнат елемент ще бъде първи премахнат.
- ❑ Механизмът се казва FIFO - First-In-First-Out.
- ❑ Приложения : printer queue, keystroke queue, etc.



Как да представим множества

Чрез асоциативни контейнери на STL-лекция 11 (12) ООП

Операции с множества:

- `set_difference()` : изчислява разлика на две множества в трето;
- `set_intersection()` : изчислява сечение на две множества в трето;
- `set_symmetric_difference()` : изчислява симетрична разлика на две множества в трето;
- `set_union()` : изчислява обединение на две множества в трето.

Как да представим множества

Пример: $A = \{a, b, c, \dots\}$

?

Чрез битове и низове. На всеки член от множеството съответства 1 бит. Ако битът = 0, елементът не е член на това множество, а ако е 1 – елементът е член на това множество.

Първите седем прости числа: $X = \{2, 3, 5, 7, 11, 13, 17\}$

Гласни в английския: $\{a, e, i, o, u\}$

Четни числа между 50 и 60

$B = \{\text{cat}, \text{dog}, \text{fox}\}$

Как да представим множества

```
// Пример за използване на асоциативен контейнер (<bitset>)
{
    /* изброим тип за представяне на дните от седмицата
    номерират дните с цифри 0-7. Последният WeekDaysNum
    е за кодиране на броя дни в седмицата
    */
    enum WeekDays { Mon, Tue, Wed, Thu, Fri, Sat, Sun, WeekDaysNum }
    // създаване на битови поредици с размерност броя (WeekDaysNum)
    bitset<WeekDaysNum> workingDays, weekendDays;
    // работен и почивен дни от седмицата се установяват в 0
        workingDays.reset(); weekendDays.reset();
    // установяване на работните дни от седмицата в 1
    workingDays.set(Mon); // понеделник
    workingDays.set(Tue); // вторник
    workingDays.set(Wed); // сряда
    workingDays.set(Thu); // четвъртък
    workingDays.set(Fri); // петък
    (продължава)
```

Как да представим множества

```
// установяване на почивните дни в 1
weekendDays.set(Sat);
weekendDays.set(Sun);
// извеждане на битовите редици в изхода
cout << "Работни дни са : " << workingDays << endl;
cout << "Почивни дни са : " << weekendDays << endl;
// операции с множества - операция допълнение
cout << "Почивни дни са неработните : " << ~workingDays << endl;
// операция сечение
cout << "Почивни & неработни : " << (workingDays & weekendDays) << endl;
// операция обединение
cout << "Почивни | неработни : " << (workingDays | weekendDays) << endl;
// операция разлика
cout << "Работни & ~непочивни : " << (workingDays & ~weekendDays) << endl;
// операция разлика
cout << "Почивни & ~неработни : " << (weekendDays & ~workingDays) << endl;
// операция симетрична разлика
cout << "Почивни ^ неработни : " << (workingDays ^ weekendDays) << endl;
(продължава)
```

Как да представим множества

```
if (workingDays.any()) {  
    for (int day = 0; day < WeekDaysNum; ++day) {  
        if (workingDays[day]) {  
            cout << "Работен е ден номер : " << day << endl;  
        }  
    }  
}  
  
if (weekendDays.any()) {  
    for (int day = 0; day < WeekDaysNum; ++day) {  
        if (weekendDays[day]) {  
            cout << "Почивен е ден номер : " << day << endl;  
        }  
    }  
}  
}
```

Как да представим множества

Особености и изисквания:

- ☐ Множествата са подредени и номерирани;
- ☐ Позицията на елемента в битовата редица е фиксирана;
- ☐ Висока ефективност на множествените операции:
 - `set_difference()` : изчислява разлика на две множества в трето с побитов $\& \sim$
 - `set_intersection()` : изчислява сечение на две множества в трето с побитов $\&$
 - `set_symmetric_difference()` : изчислява симетрична разлика на две множества в трето с побитов $\wedge$
 - `set_union()` : изчислява обединение на две множества в трето с побитов $|$
- ☐ При вмъкване/изтриване на елемент подредбата се променя:
 - ниска ефективност при чести добавяния/изтривания;

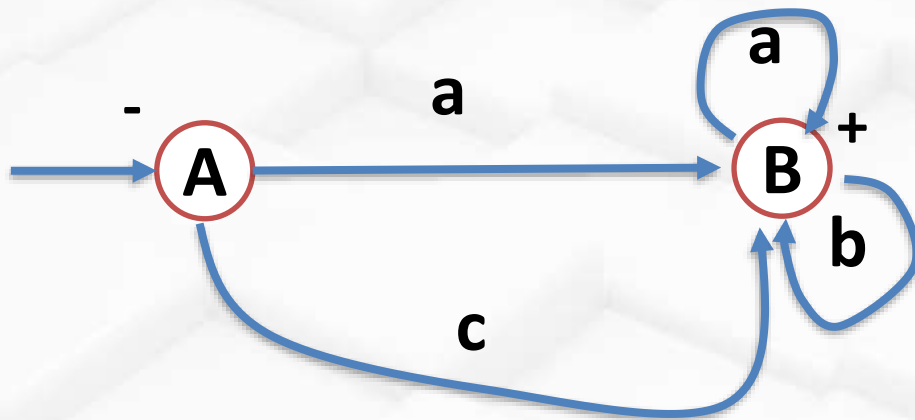
Как да представим отношения и функции

Използват се обобщени алгоритми в комбинация с

Чрез STL-функционални обекти ООП:

- ☐ Клас функции;
- ☐ Lambda;
- ☐ Статични;
- ☐ Глобални-не бива да се използват

Как да представим краен автомат в паметта?

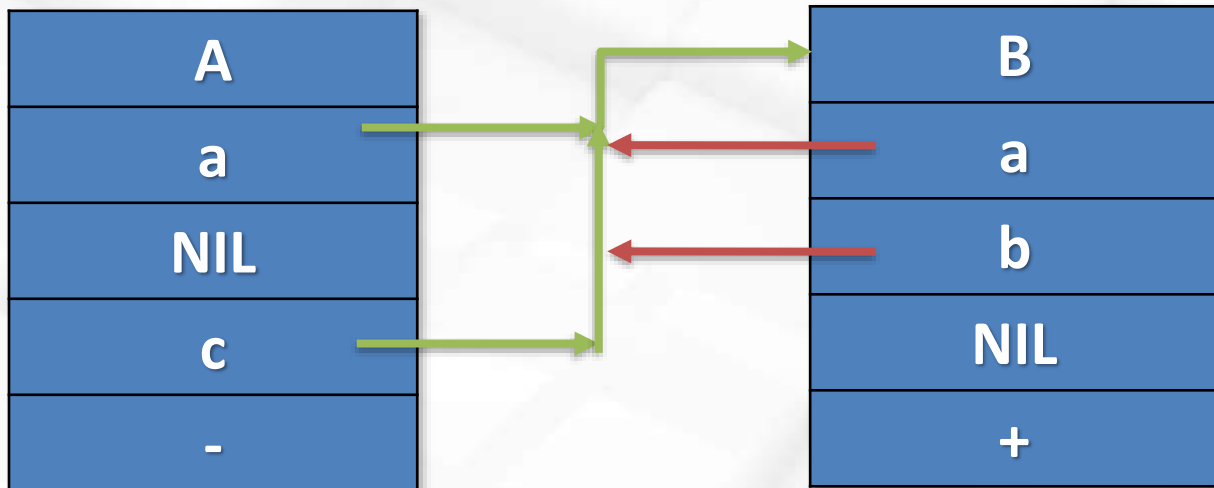
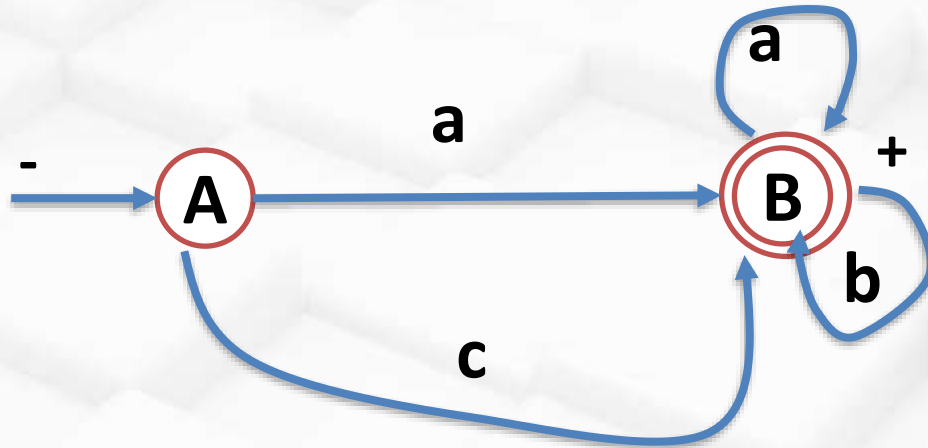


Всеки възел се представя като списъчна структура, състояща се от:

- име - string
- указатели – толкова указатели, колкото са буквите от азбуката на езика.
- информация за типа на възела.
 - вход
 - + изход
 - 0 друг

име	
a	→
b	→
c	→
info	

Представяне в паметта



Представяне на МП

Също както краен ориентиран граф + допълнително означение кои са места кои са преходи.

Списъчна структура за

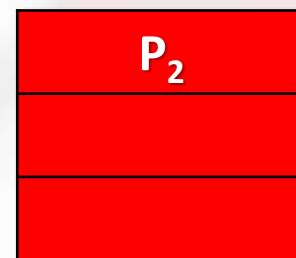
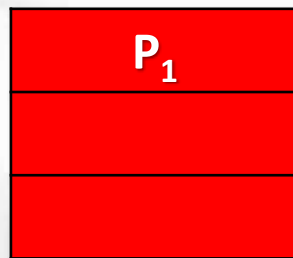
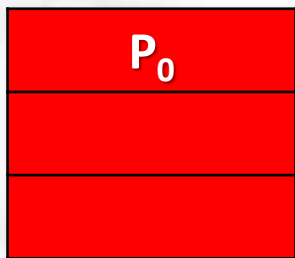
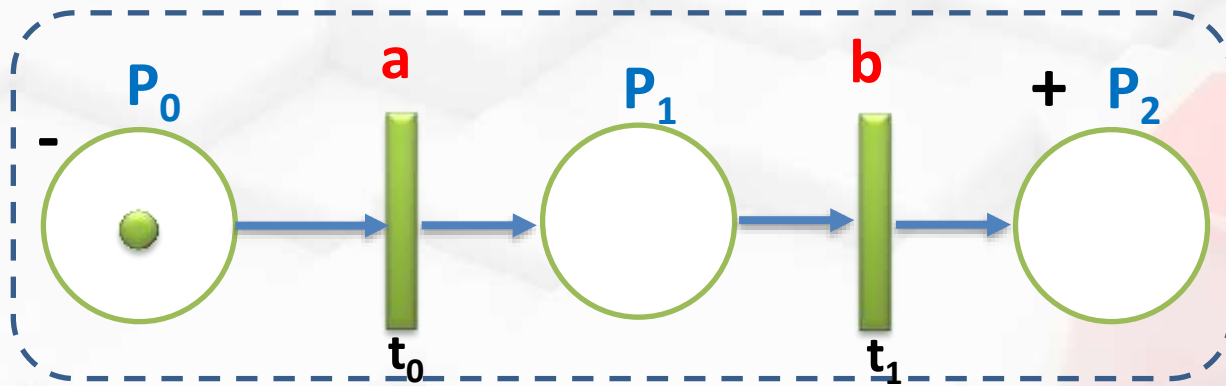
МЯСТО

име на място
брой марки
тип място

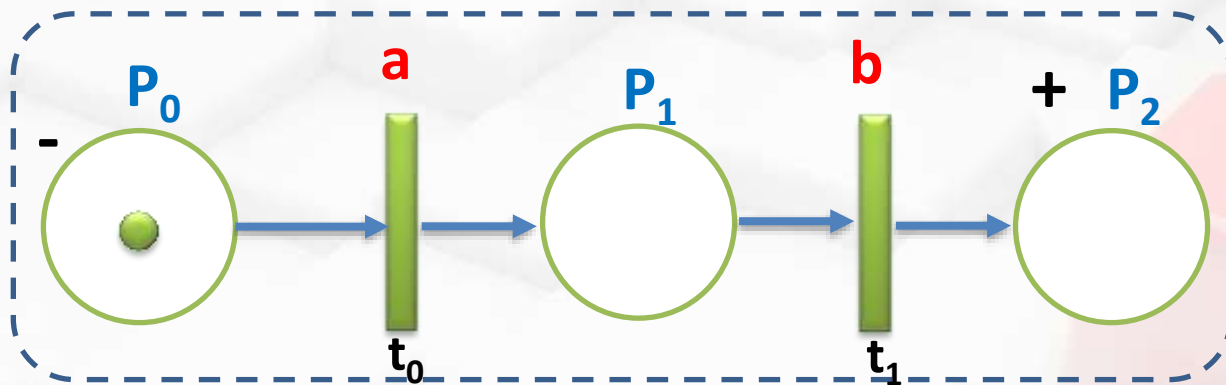
преход

име на преход
указатели към входни места
указатели към изходни места
разпознаван символ

Пример



Пример

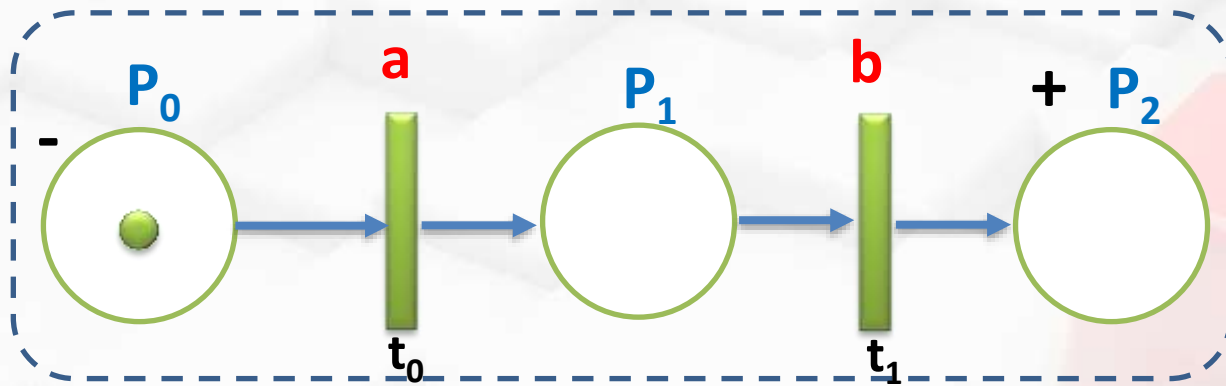


P_0
1

P_1

P_2

Пример

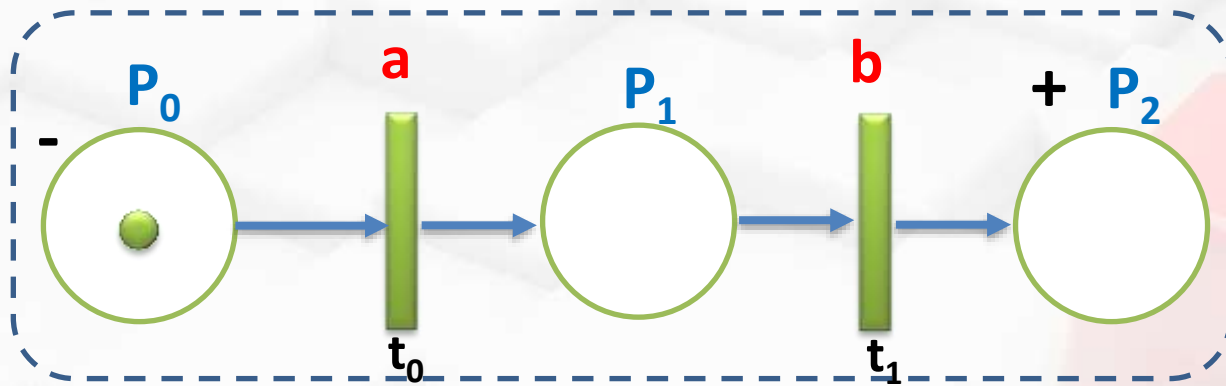


P_0
1

P_1
0

P_2

Пример

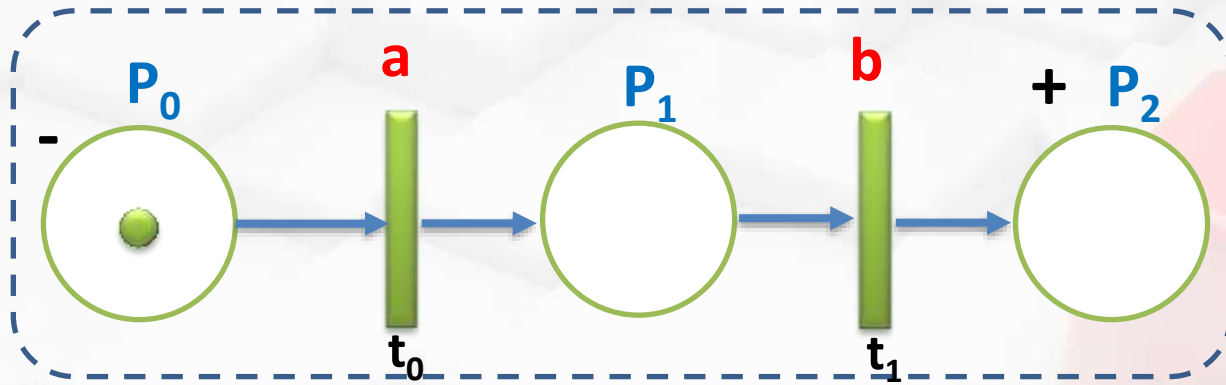


P_0
1

P_1
0

P_2
0

Пример

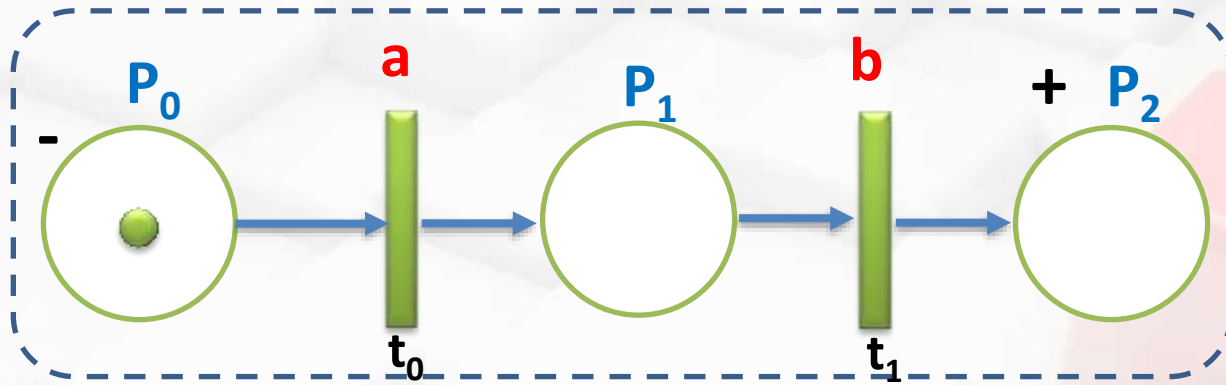


P_0
1
-

P_1
0

P_2
0

Пример

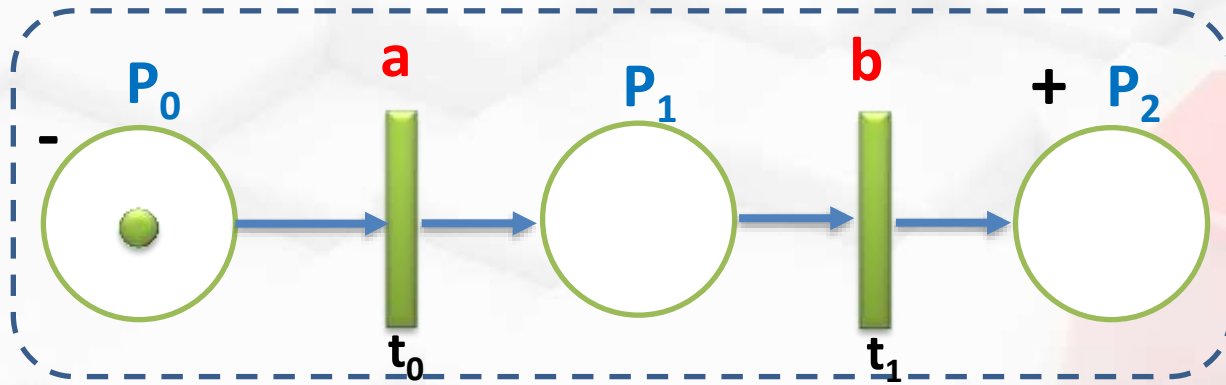


P_0
1
-

P_1
0
0

P_2
0

Пример

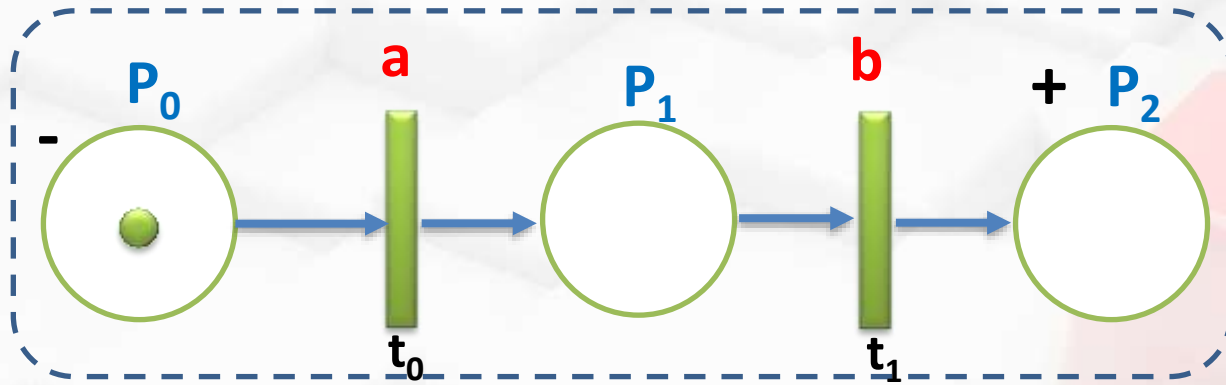


P_0
1
-

P_1
0
0

P_2
0
+

Пример



P_0
1
-

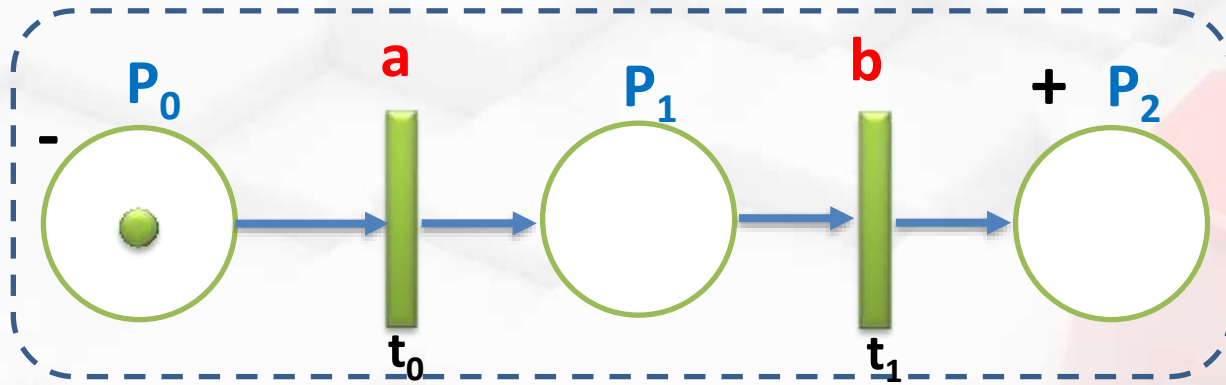
P_1
0
0

P_2
0
+

t_1

t_2

Пример



P_0
1
-

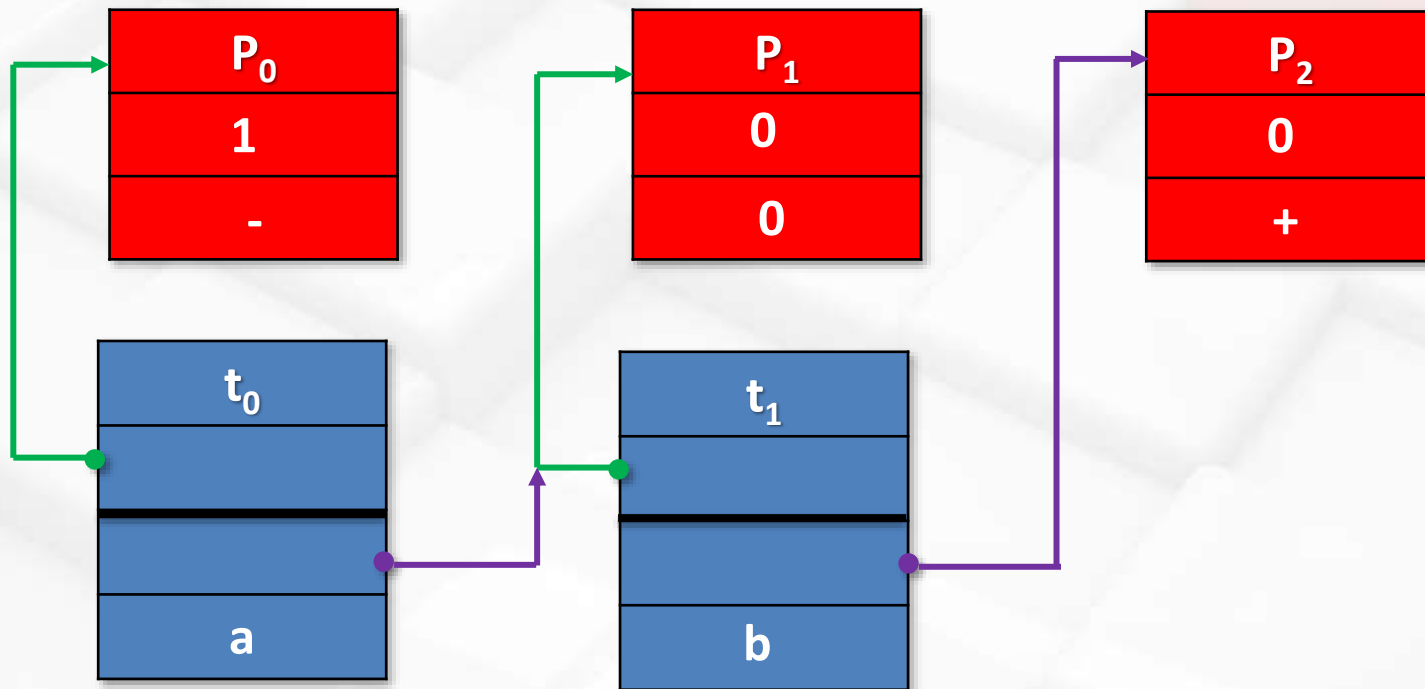
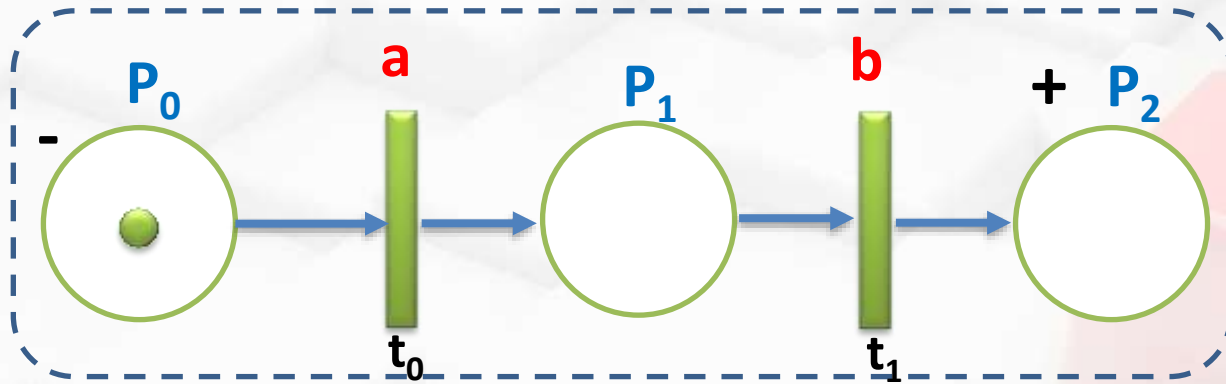
P_1
0
0

P_2
0
+

t_1
a

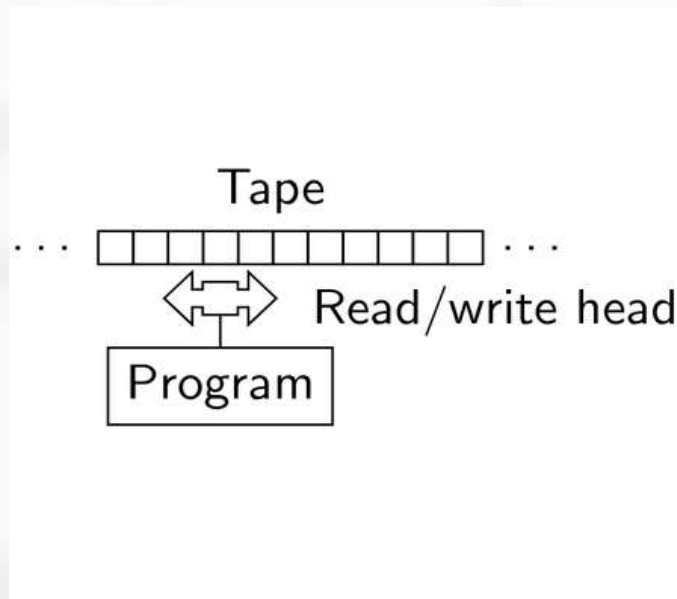
t_2
b

Пример



Представяне на МТ

Библиотека STL е Фон Нойманов автомат (МТ)



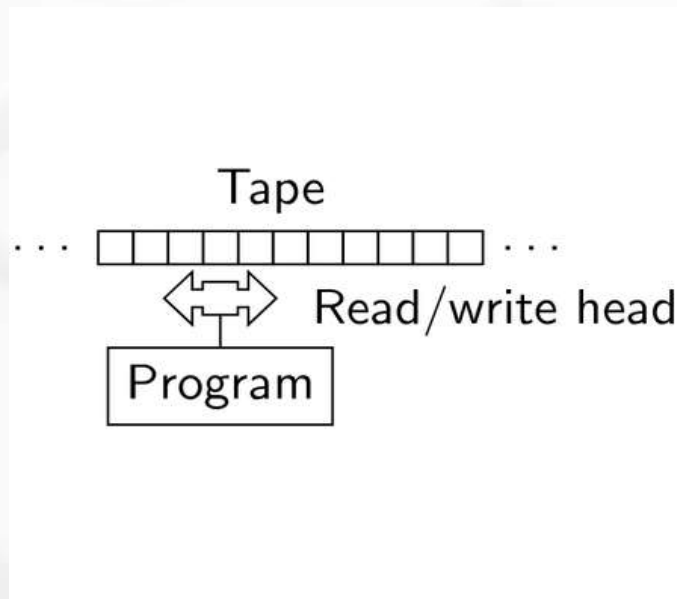
Контейнер (Container)

Итератор (Iterator)

Алгоритъм (Algorithm)

Представяне на МТ

STL стек - ограничен вариант по операции (Iterator, Program)



Контейнер (Container)

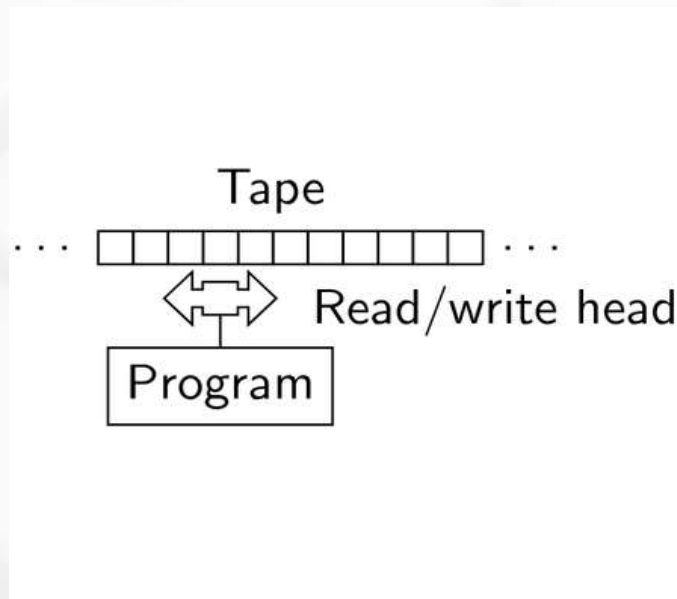
Итератор (Iterator)

Алгоритъм (Algorithm)

Представяне на МТ

STL броячен автомат (стек с ограничено действие):

- Не съхранява реално а брой еднакви букви....



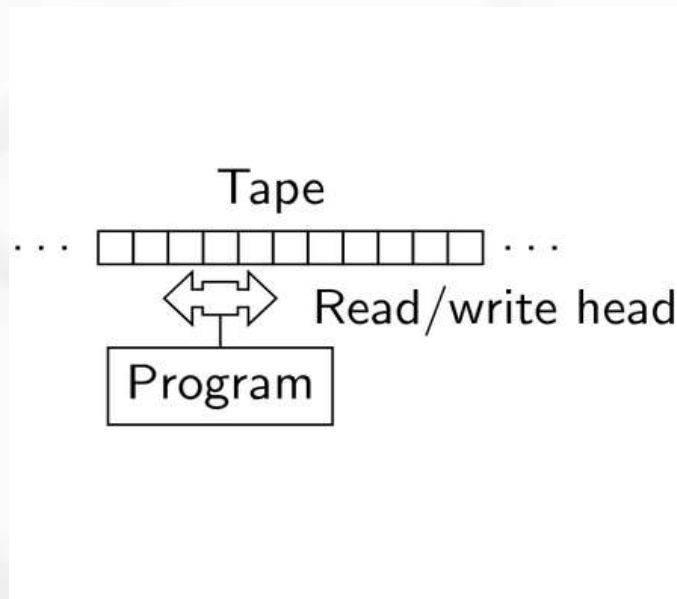
Контейнер (Container)

Итератор (Iterator)

Алгоритъм (Algorithm)

Представяне на МТ

STL опашка - ограничен вариант (Iterator, Program)



Контейнер (Container)

Итератор (Iterator)

Алгоритъм (Algorithm)

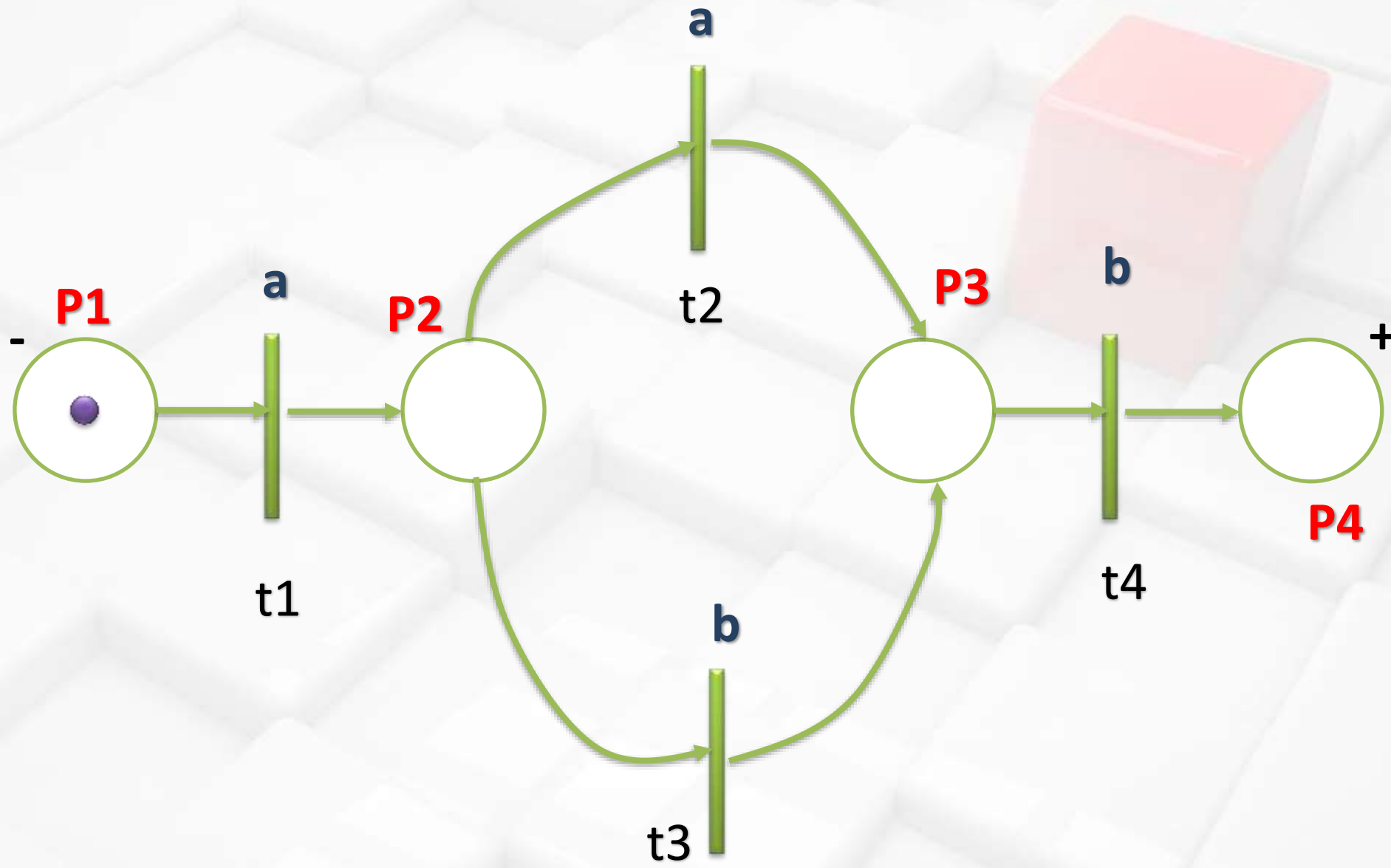
Съдържание

11.1. Представяне на моделите в паметта.

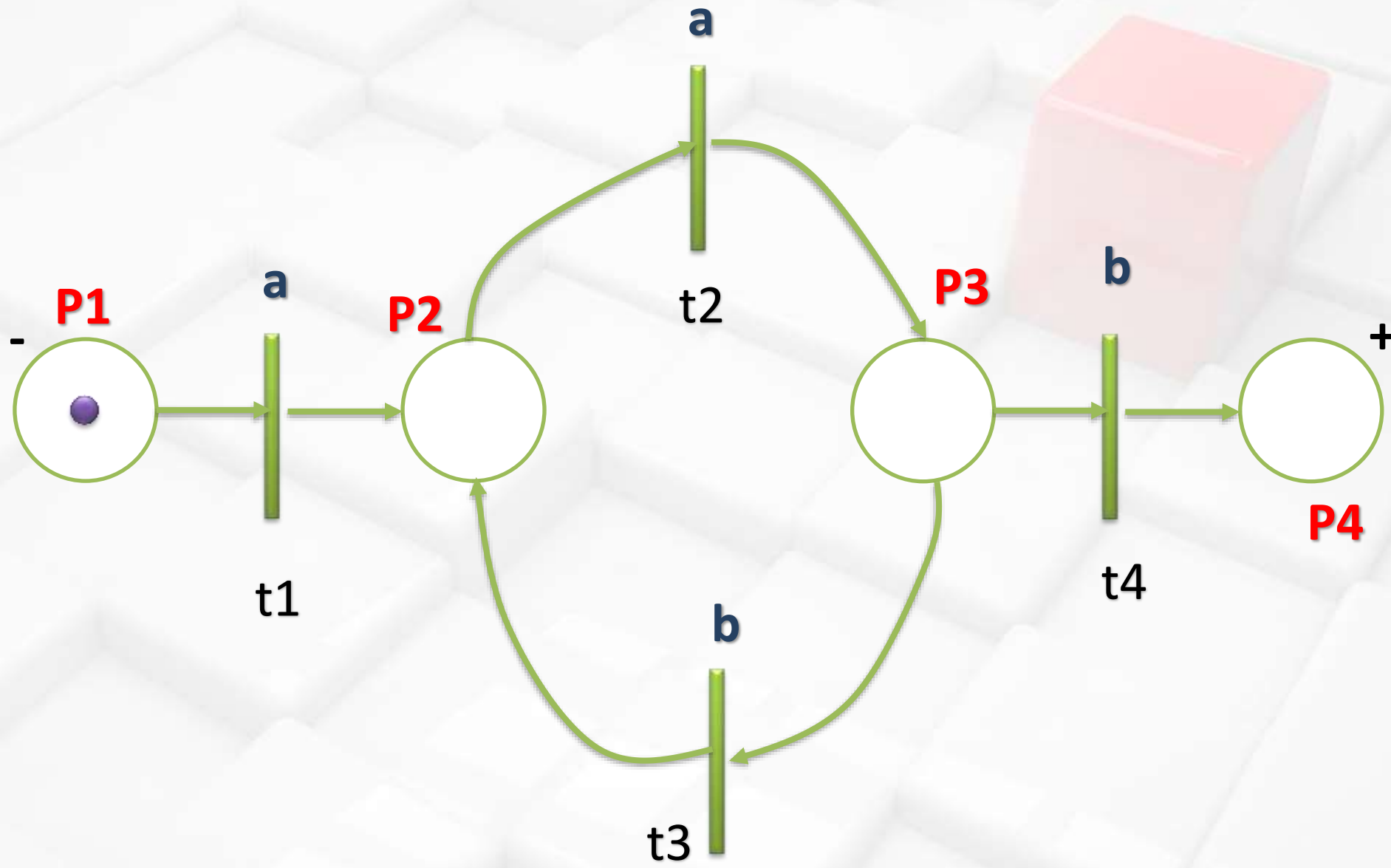
11.2. Задачи

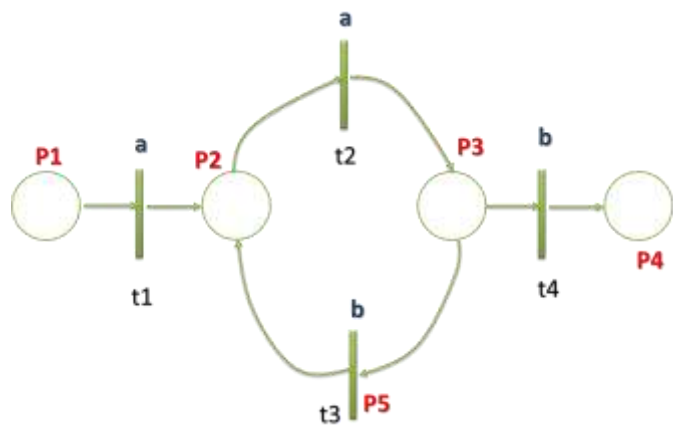
11.3. Моделиране на паралелни процеси

Задача 1. Дадена е обозначена ММП. Представете
чрез израз, граматика, списъчна структура

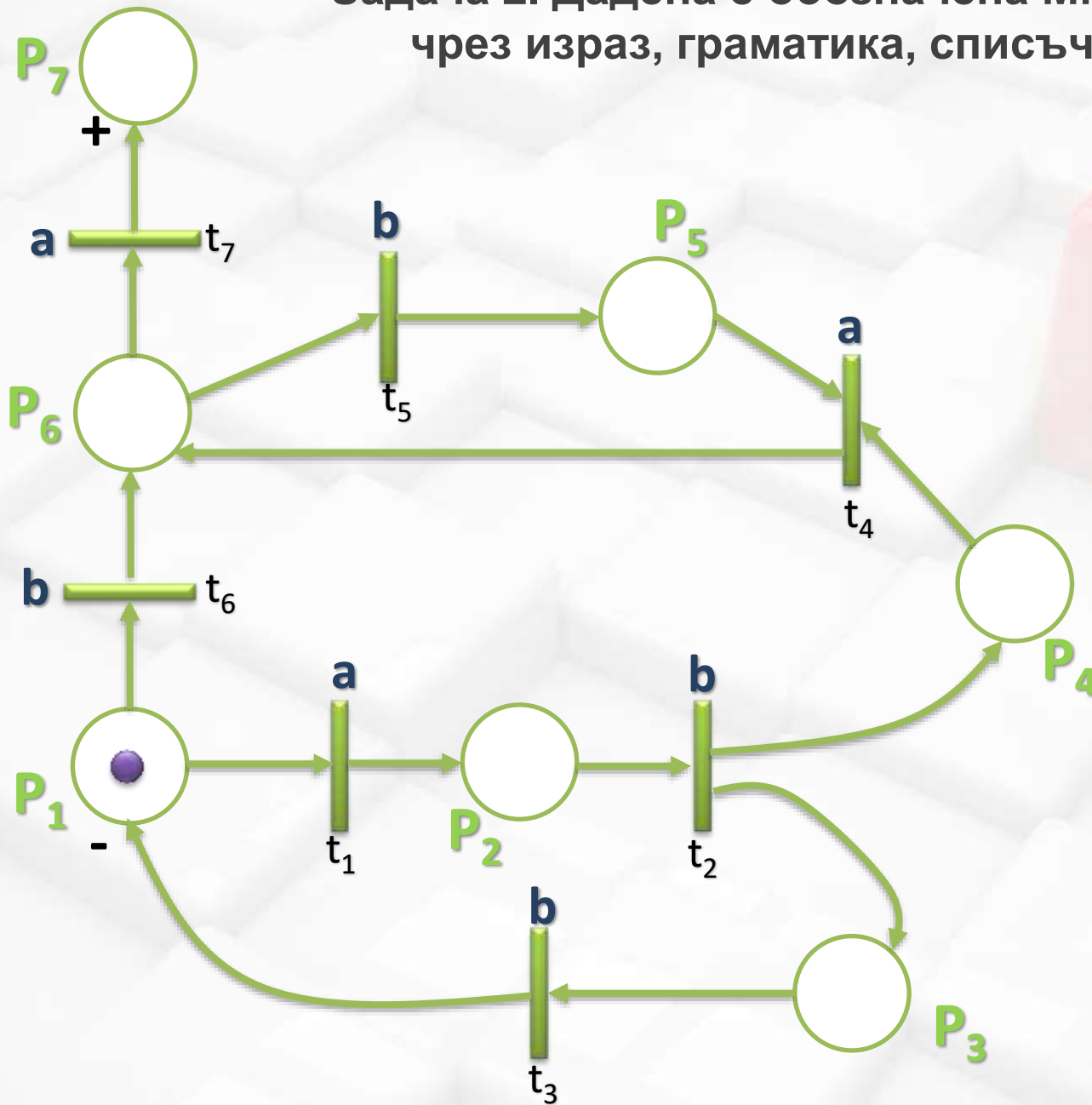


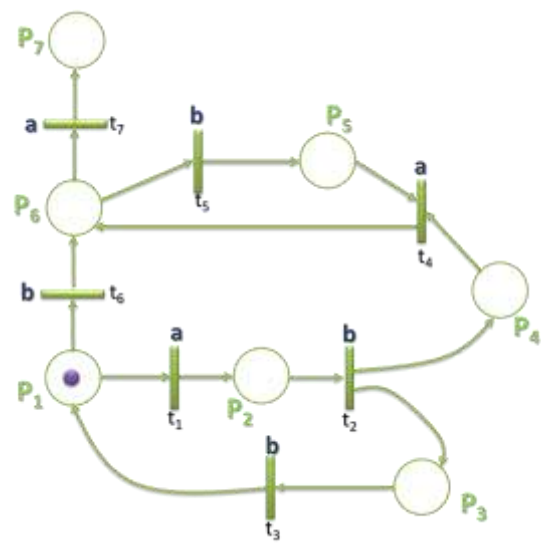
Задача 1. Дадена е обозначена ММП. Представете
чрез израз, граматика, списъчна структура





Задача 2. Дадена е обозначена ММП. Представете
чрез израз, граматика, списъчна структура





Приложения на ММП – маркирани мрежи на Петри.

Видове отношения между процеси.

Видове зависими отношения.

12.1. Моделиране на паралелни процеси (ПП).

12.2. Взаимодействие между паралелни процеси.

12.3. Координиране и разпределение на ресурси между ПП.

- ☐ семафор

- ☐ взаимно изключване (**mutex**)

12.4. Примери

Видове отношения

Видове отношения между паралелните процеси (ПП) във времето:

- ❑ Независими ПП-между тях няма причинно-следствена връзка (самостоятелни);
- ❑ Зависими ПП-между тях има поне 1 причинно-следствена каузална връзка (отношение):
 - **каузално отношение**-преход в определено състояние на единия процес е зависимо от достигане на състояние на втория;
 - процесите се синхронизират в каузалната връзк(а/и);
 - работят паралелно (асинхронно) в останалото време.

Видове зависими отношения

Видове зависими отношения между паралелните процеси (ПП) във времето:

- ❑ Коопериращи отношения, постановки:
 - N паралелни задачи върху самостоятелни устройства, всяка с различно време, приключват общата задача, когато най-продължителната завърши;
 - Отношения „генератор-консуматор“, между A и B:
 - не настъпват паралелно;
 - A е безусловно;
 - B зависи(настъпва) след A.
- ❑ Конкуриращи се отношения

Видове зависими отношения

Конкурентност и конфликт:

- ❑ N паралелни задачи използват общ ресурс при паралелната работа:
 - Критичен ресурс;
 - Управление на ресурса;

Дефиниции

Едновременни

дефинира процеси, които могат да се приложат едновременно.

Паралелни

едновременна програма, която е направена за паралелен хардуер.

Разпределени

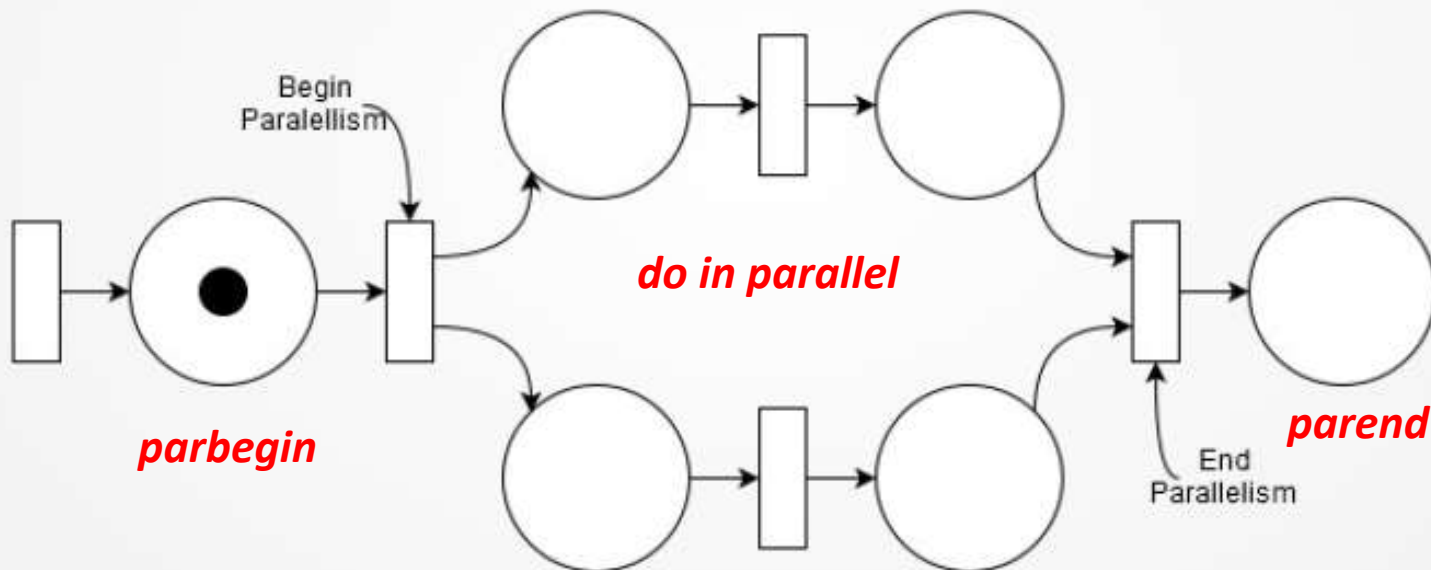
паралелна програма, която е направена за изпълнение на мрежа от независими процесори, които нямат обща памет.

недетерминираност

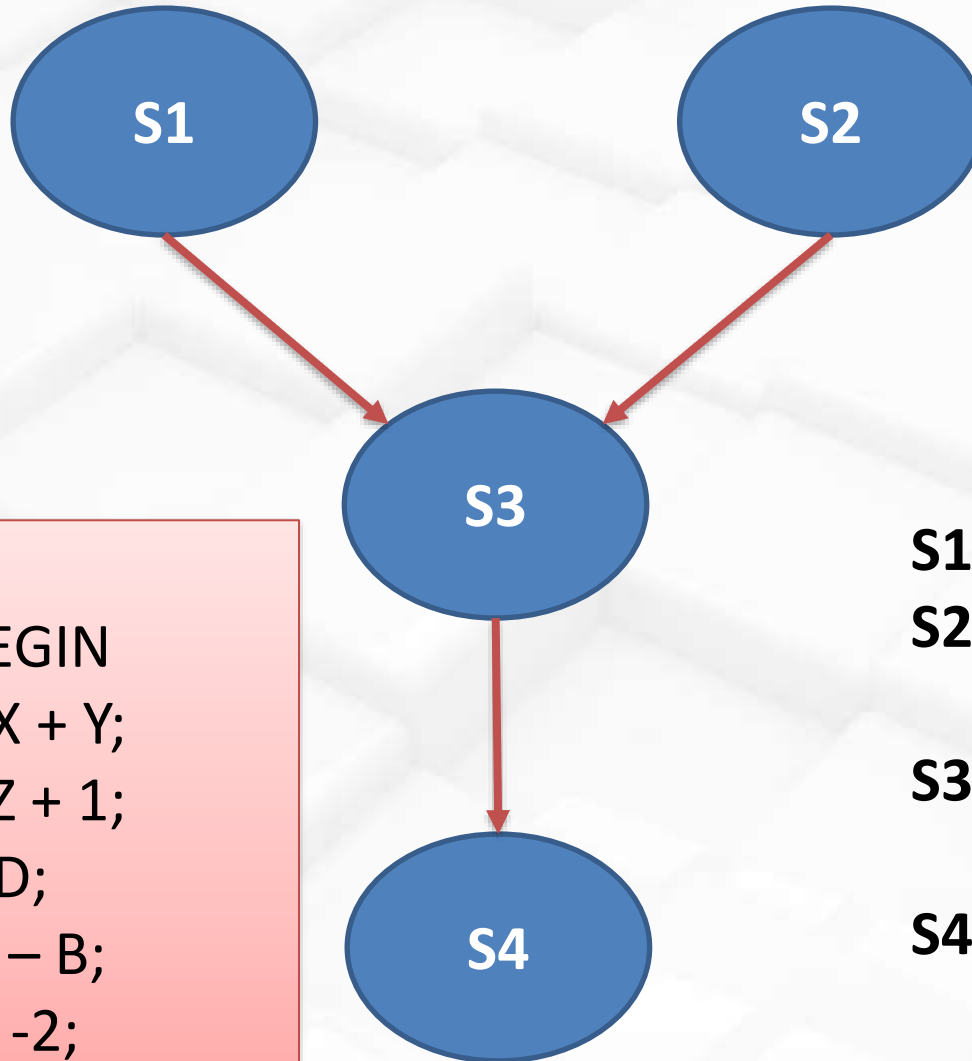
Синхронизация и комуникация между паралелни процеси

Коопериращи отношения:

Обща задача, която в даден етап може да се раздели на независими части (**task**), които да се изпълнят на самостоятелни физически или логически устройства (**threads**), след завършване на всички (най-продължителната независима задача), продължава в едно единствено устройство. Може да се моделира това поведение с ОММП:



Демонстрация



```
Begin  
  PARBEGIN  
    A = X + Y;  
    B = Z + 1;  
  PAREND;  
    C = A - B;  
    D = C - 2;  
End
```

S1 → **A = X + Y;**

S2 → **B = Z + 1;**

S3 → **C = A - B;**

S4 → **D = C - 2;**

```

omp_set_num_threads(4);
cout << omp_get_max_threads();

#pragma omp parallel for \
shared(GantryTransform, EulerSystem, image, delta_image, irr_geom, detector_center, l, ray_step, SOURCE, ax, ay, az,
private(ww, hh, raylength, bx, by, bz, t1, t2, offset_t1, offset_t2, x_m, y_m, z_m, temp_a, temp_b, temp_c, att_coef)
for(hh = 0; hh < irr_geom.height; hh++)
{
    cout << "Current hh = " << hh << endl;
    for(ww = 0; ww < irr_geom.width; ww++)
    {
        double temp[3]; temp[0] = 0; temp[1] = 0; temp[2] = 0;
        double *p; p = &temp[0];
        double DETECTOR[3];

```

```

            exponent = ((double)((ii-0-10)*(ii-0-10)) + (double)((jj-0-10)*(jj-0-10)))/(2.0*std_sigma*std_sigma);
            #pragma omp atomic
            image[ii+hh-10][jj+ww-10] += (exp(-exponent))*line_int/(sum_exponent);
            #pragma omp atomic
            delta_image[ii+hh-10][jj+ww-10] += (exp(-exponent))*delta_int/(sum_exponent);
        }
    }

    // image[ww][hh] = line_int;
    // delta_image[ww][hh] = delta_int;
} // end of for for ww

    counter_columns--;
    cout << " " << (((double)counter_columns)/((double)irr_geom.height))*100.0 << "%, ";
} // end for for hh
cout << endl;

#pragma omp barrier
#pragma omp end parallel
}

```

Взаимодействие между процесите

Всички паралелно работещи програми включват взаимодействие между процесите.

Защо?

- ☐ Процесите се състезават за изключителен (**exclusive**) достъп до споделени (**shared**) ресурси- физически устройства или данни;
- ☐ Процесите комуникират помежду си за да обменят данни.
- ☐ И в двата случая е необходимо дадените процеси да **синхронизират** тяхното изпълнение, за да избегнат **конфликт** при получаване на данни, и при **контакт**, когато се обменят данни.

Как взаимодействат процесите?

Чрез

- ☐ споделени (**shared**) променливи;
- ☐ Изпращане на съобщения от един процес на друг („**Message passing**“).

Координиране и разпределение на ресурси между паралелните процеси

❑ **Управление на ресурси** - процес, който иска да използва споделени (**shared**) ресурси (например **printer**) трябва първо да получи разрешение за тези ресурси.

❑ **Комуникация**

❑ **Синхронна** - процесите се синхронизират за обмяна на данни;

❑ **Асинхронна** - процес, който осигурява данни може да се оттегли за получаване на данни, без това да доведе до забавяне в случай, че процесът на получаване се забави много.

Проблеми

MUTEX – mutual exclusion – взаимно изключване

Deadlock - блокада

Неопределено чакане

Проблеми

MUTEX – mutual exclusion

Deadlock

Неопределено чакане



Критичен регион

Някои операции при паралелните процеси се провалят да произведат дадения ефект, ако те са приложени върху два или повече процеса едновременно. Такъв код съдържа **критичен регион (секция)**.

Ако някой процес се намира в този регион, всички останали трябва да нямат достъп до този регион, докато приключи първият процес.

Когато конструираме паралелни програми, трябва да дефинираме къде има такова **mutual exclusion** и да го контролираме.

Защо е необходимо взаимно изключване (mutual exclusion)

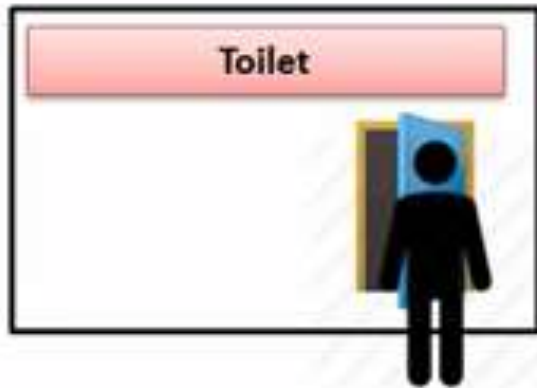
Пример:

$X = X + 1;$

mutex



mutex



mutex



mutex



Проблеми

MUTEX – mutual exclusion

Deadlock – процес, който чака за събитие, което никога няма да се случи.

Неопределено чакане

Проблеми

MUTEX – mutual exclusion

Deadlock

Неопределено чакане – един процес е неопределено отложен, ако той закъснява поради чакане на събитие, което може и да не се случи.

Взаимодействие между процесите чрез използване на споделени променливи

Използват се в критични секции.

Необходима е защита чрез използване на:

- ☐ **Status променливи** – напр. memory bit, който индикира дали или не регионът в момента е окупиран от друг процес.
- ☐ **Семафори** – цяло число, високо ниво на абстракция на механизма на status променливата. Входът и изходът от критичната секция се контролира от P и V операциите, които означават:
 - ☐ P – wait if necessary
 - ☐ V - signal

Семафор

Семафорът се контролира от $P()$, $V()$.

$P()$ – използва се да намали стойността на семафора.

$V()$ – използва се да увеличи стойността на семафора.

Дефиниция за P(), V()

P(S) :

S=S-1

if (S<0)

wait on S

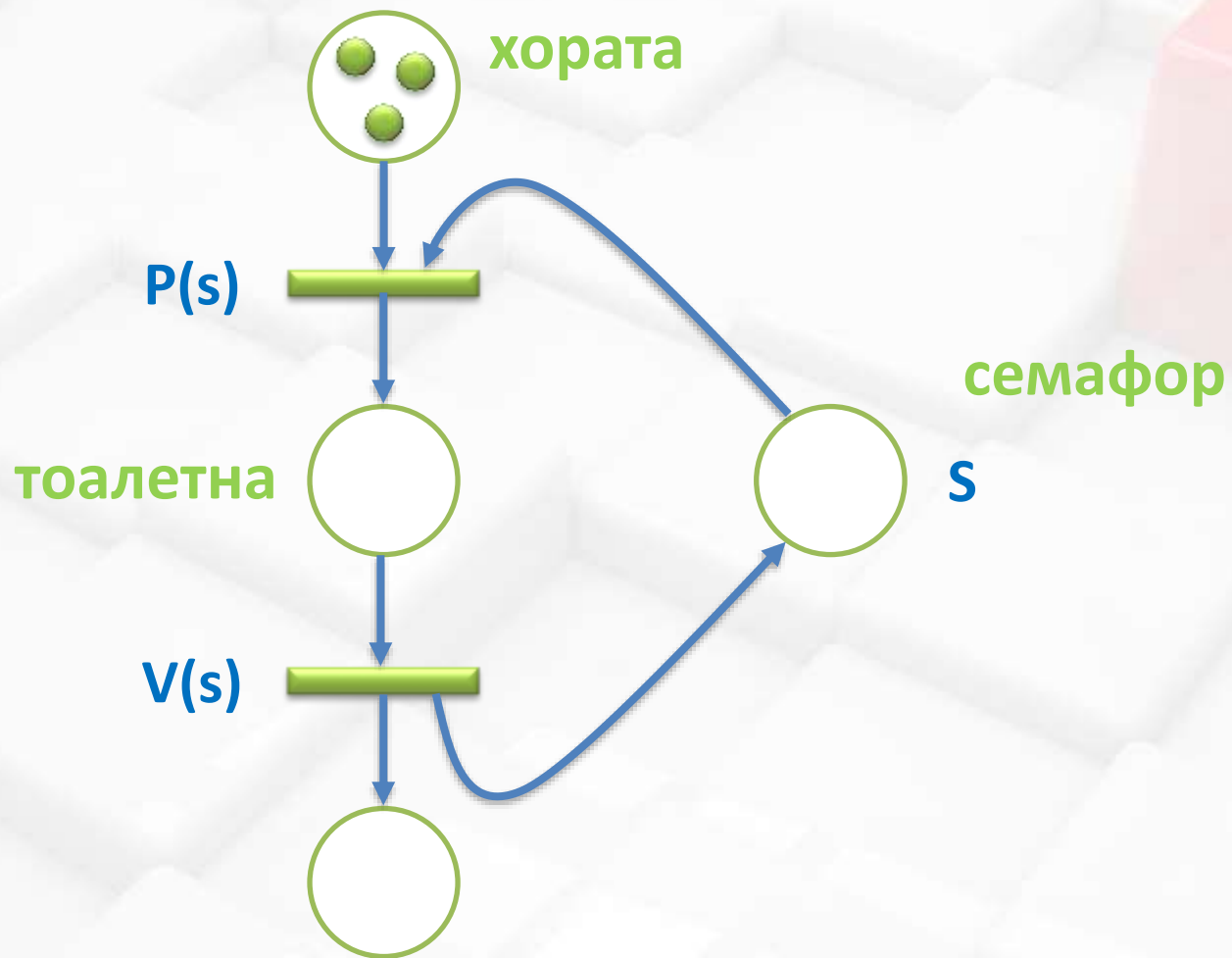
V(S) :

S=S+1

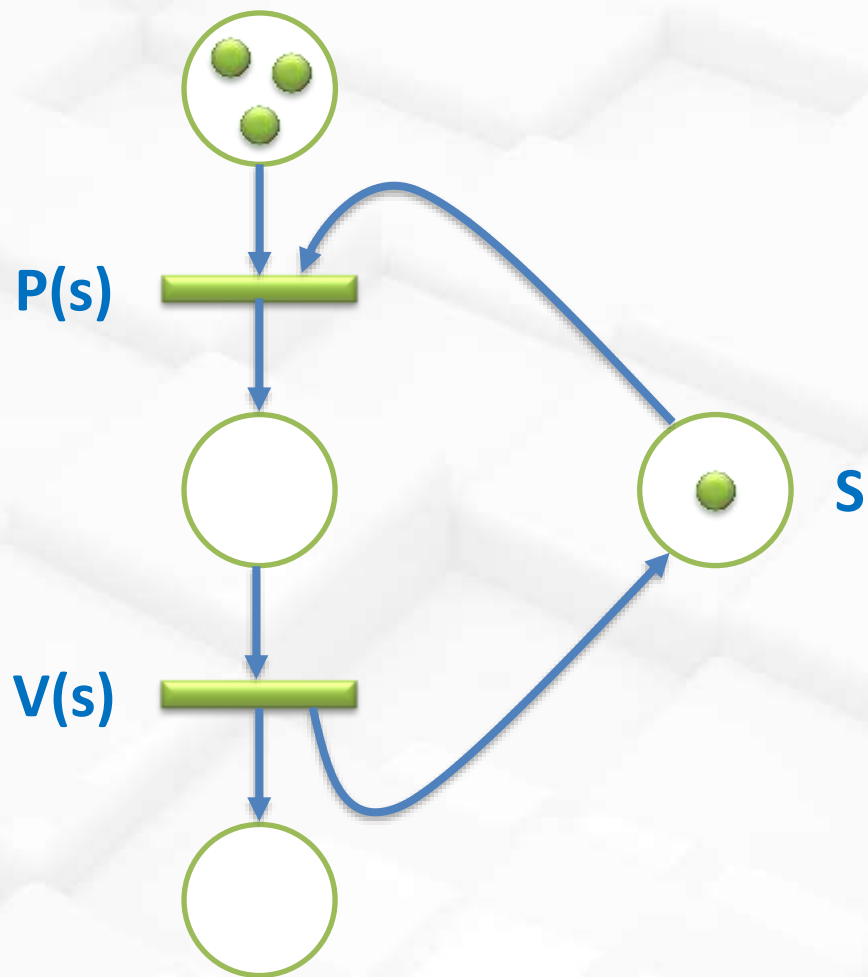
if (threads waiting on S)

wake one

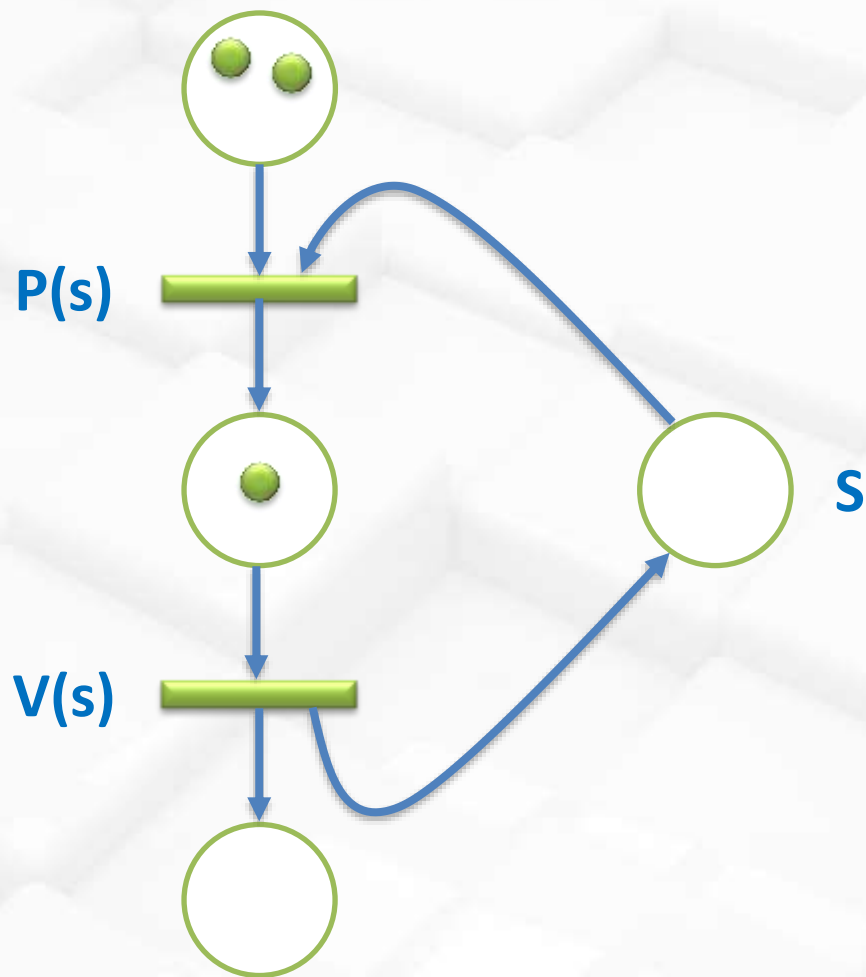
Семафор



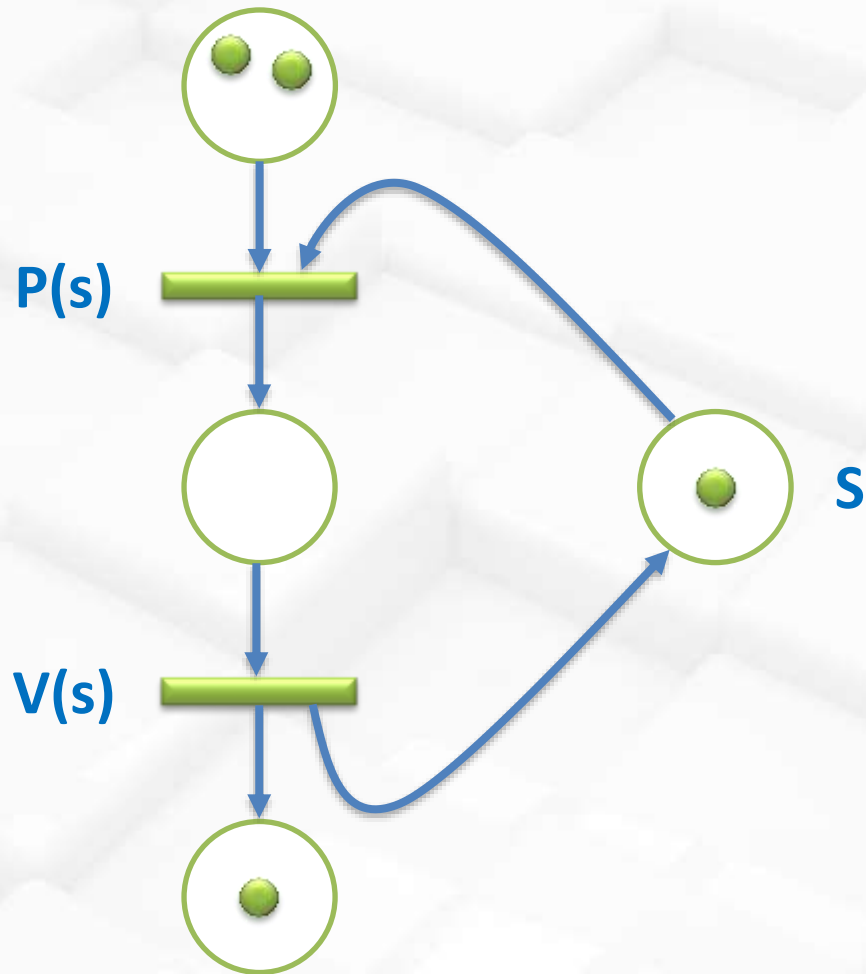
Семафор



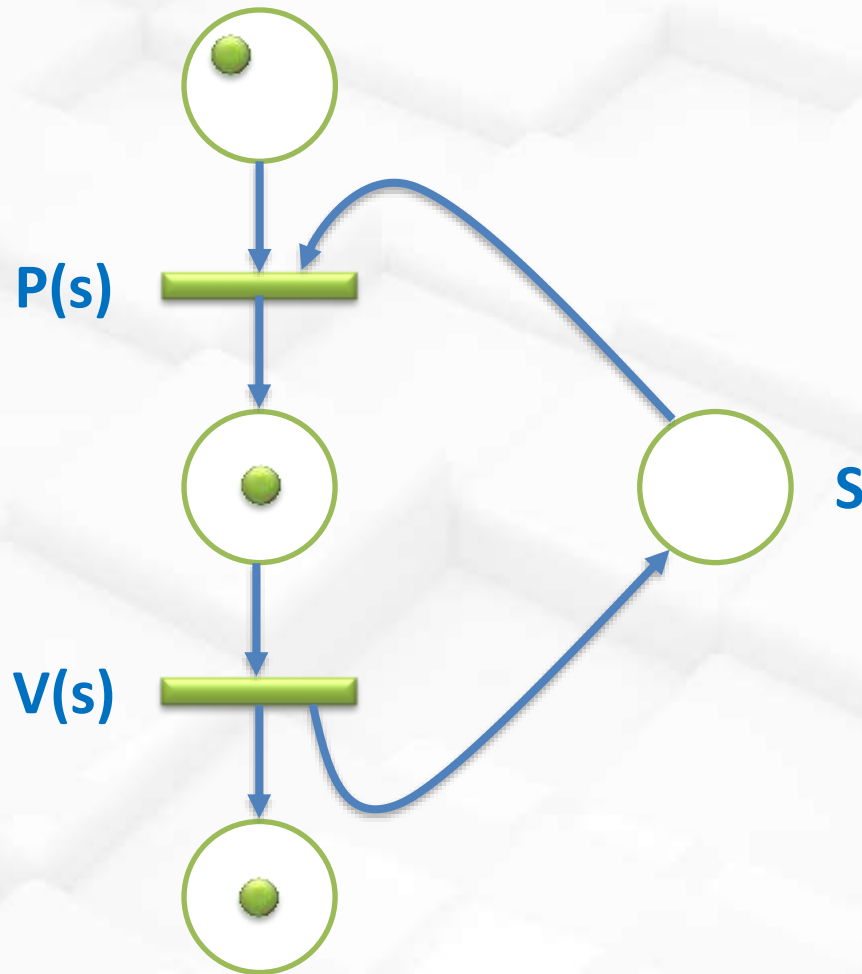
Семафор



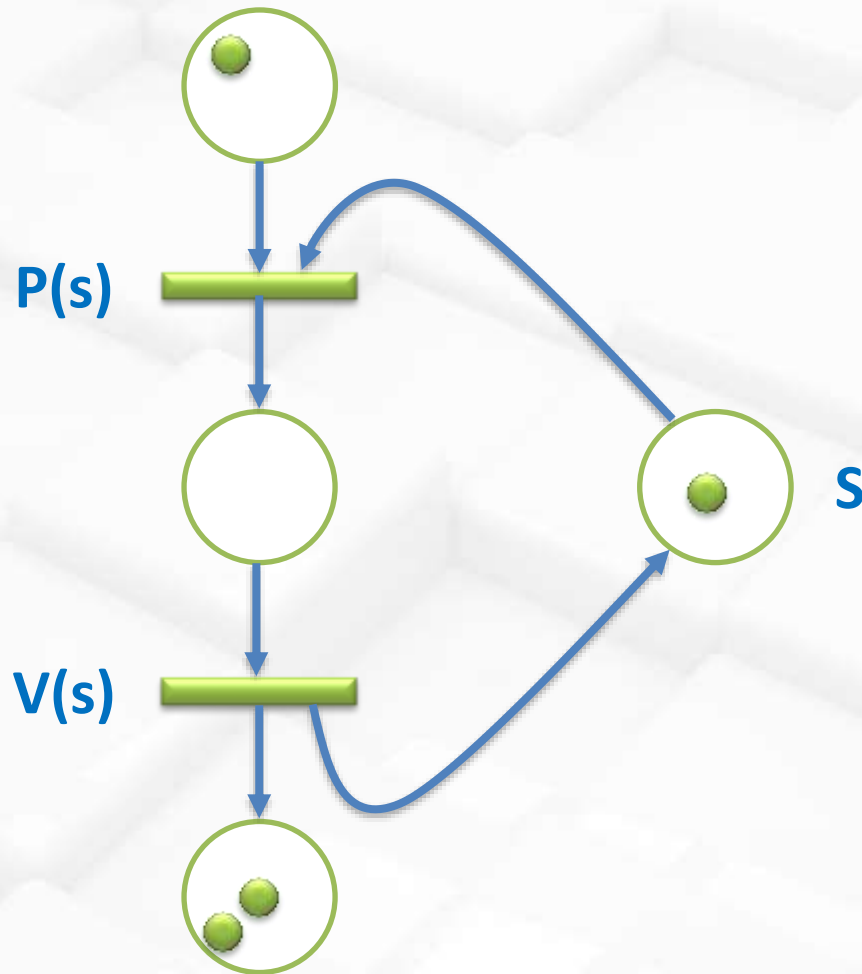
Семафор



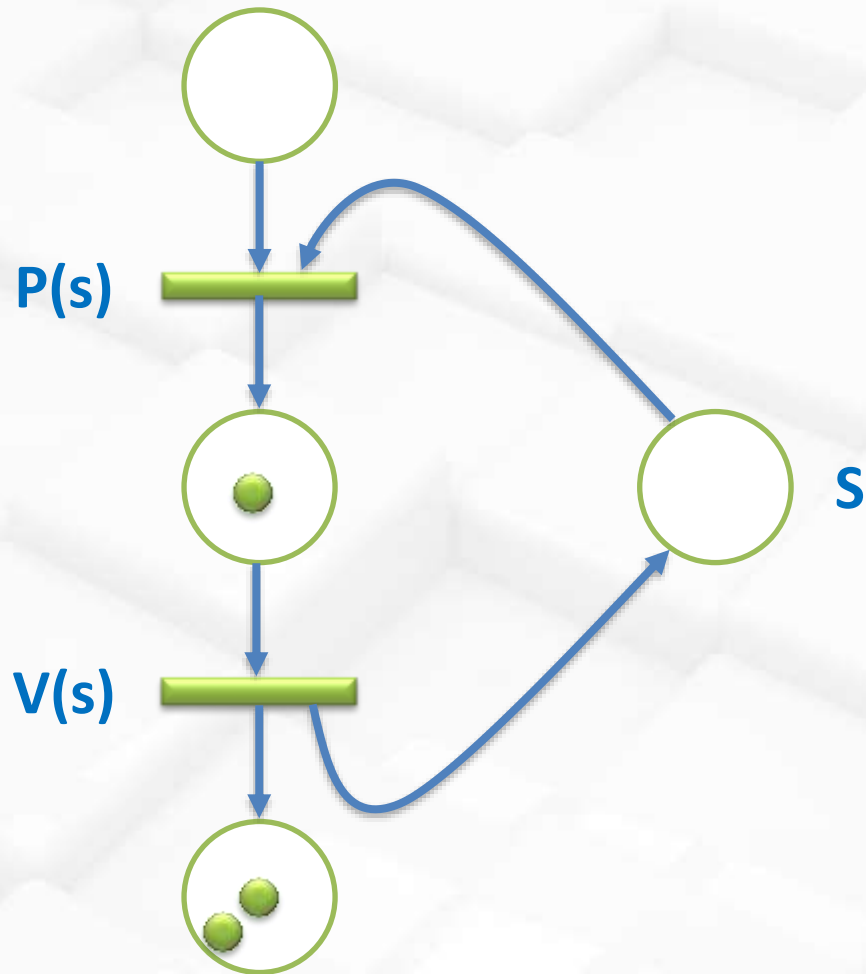
Семафор



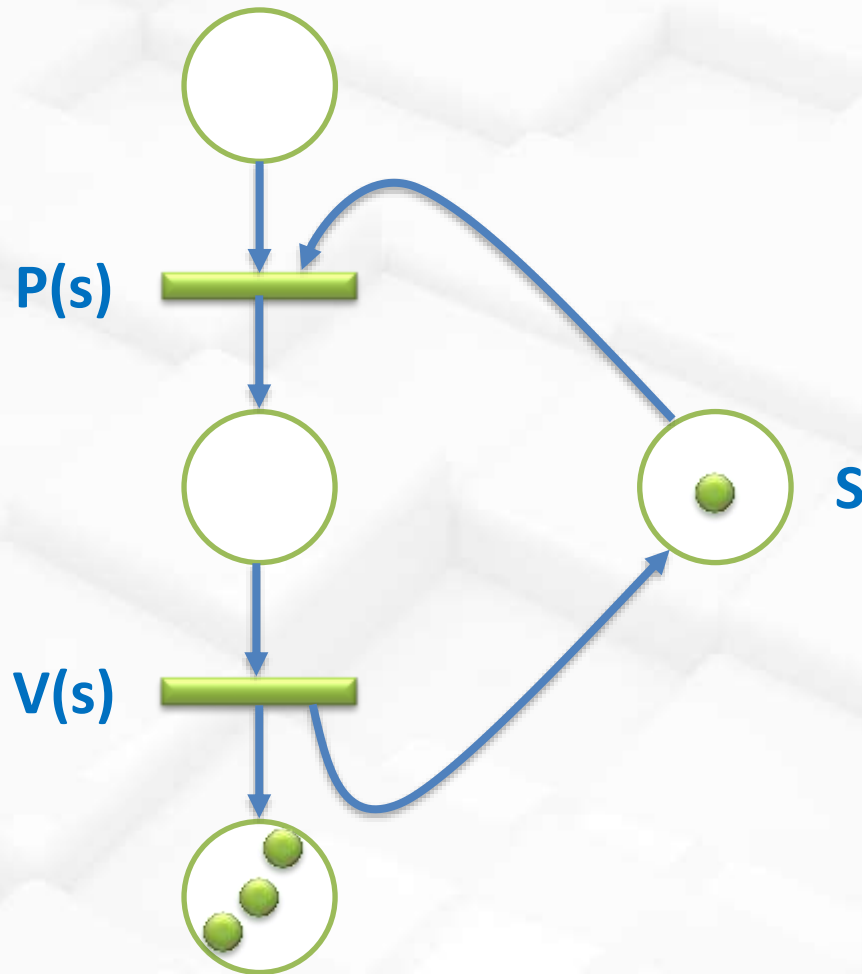
Семафор



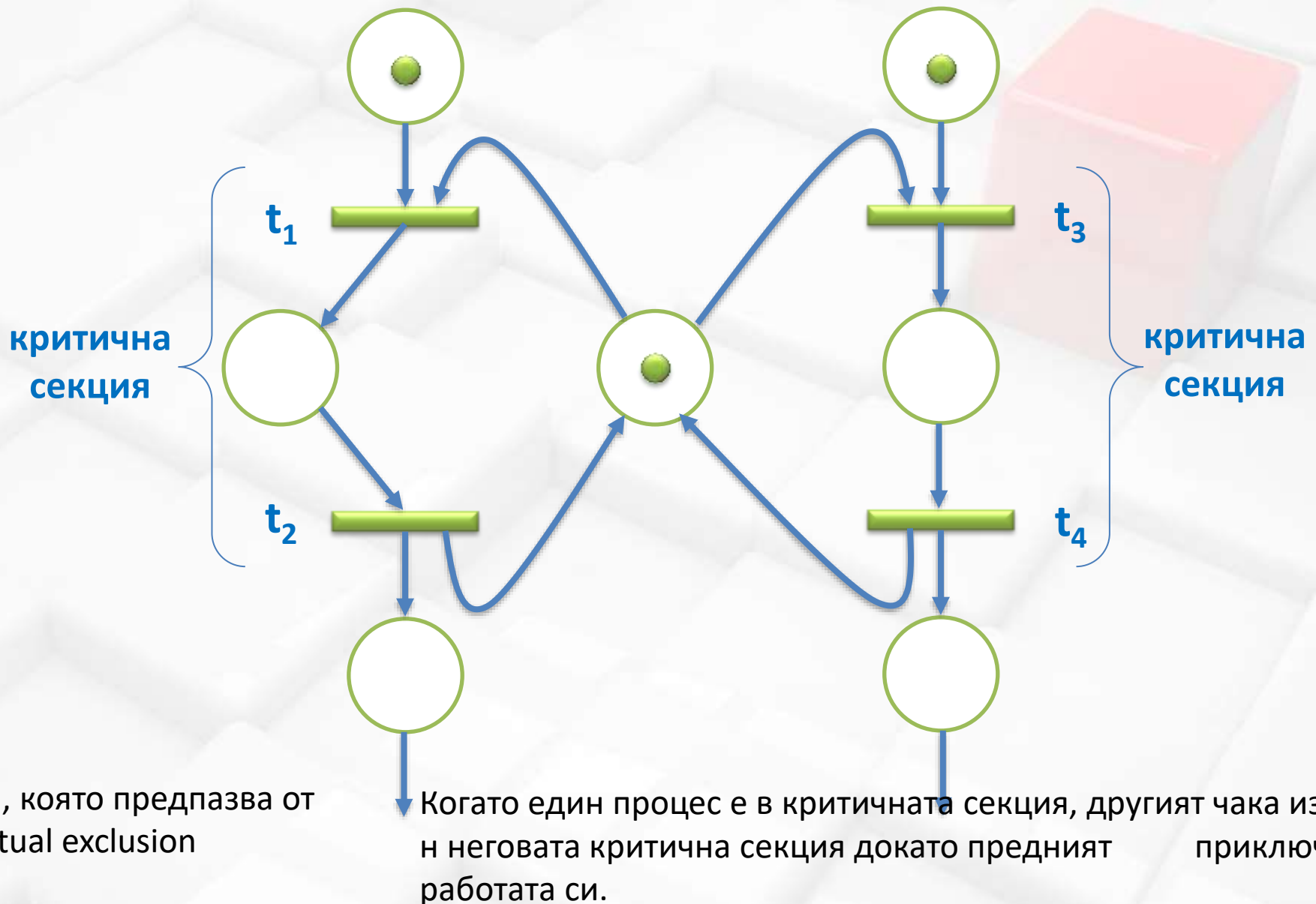
Семафор



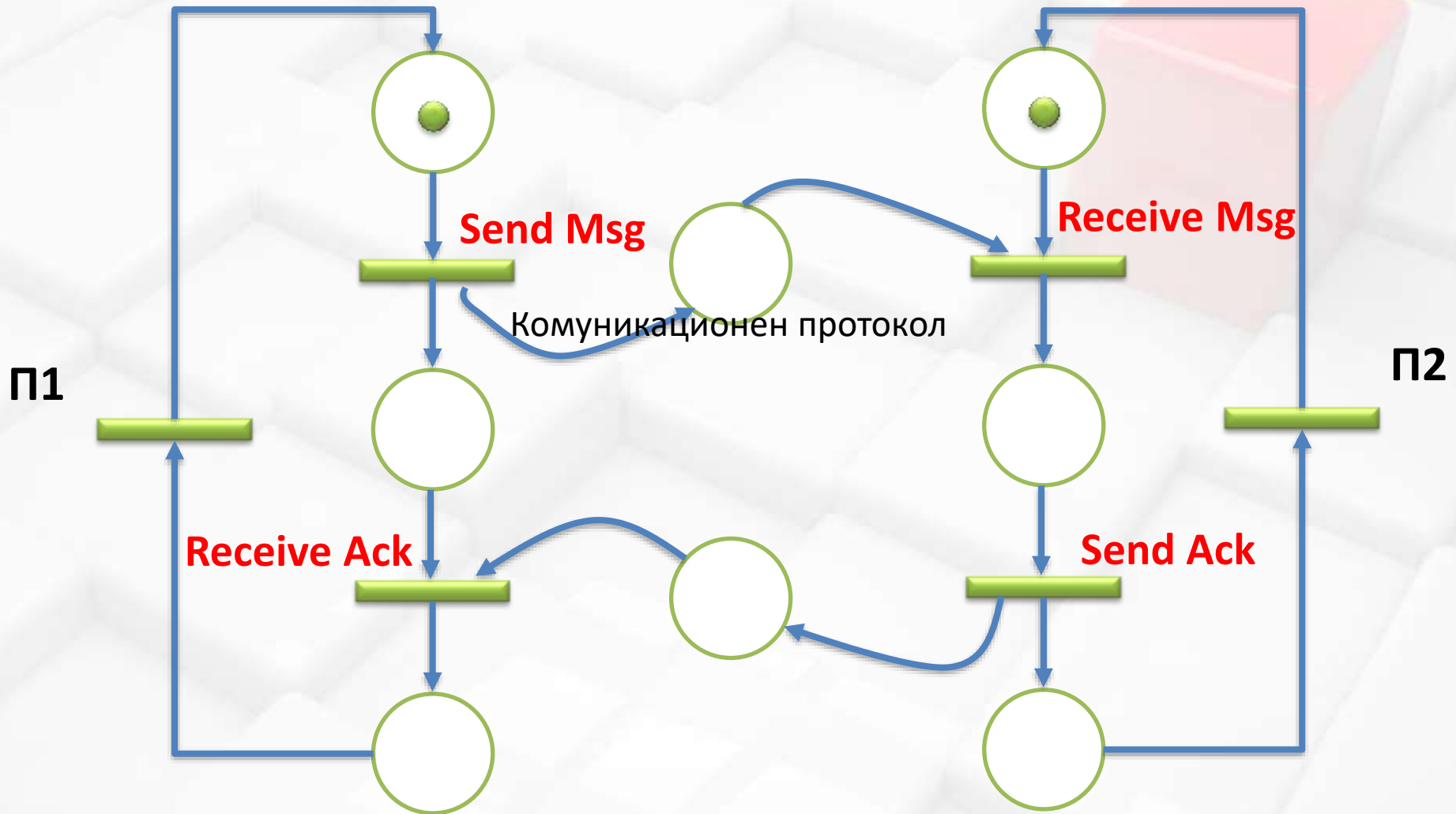
Семафор



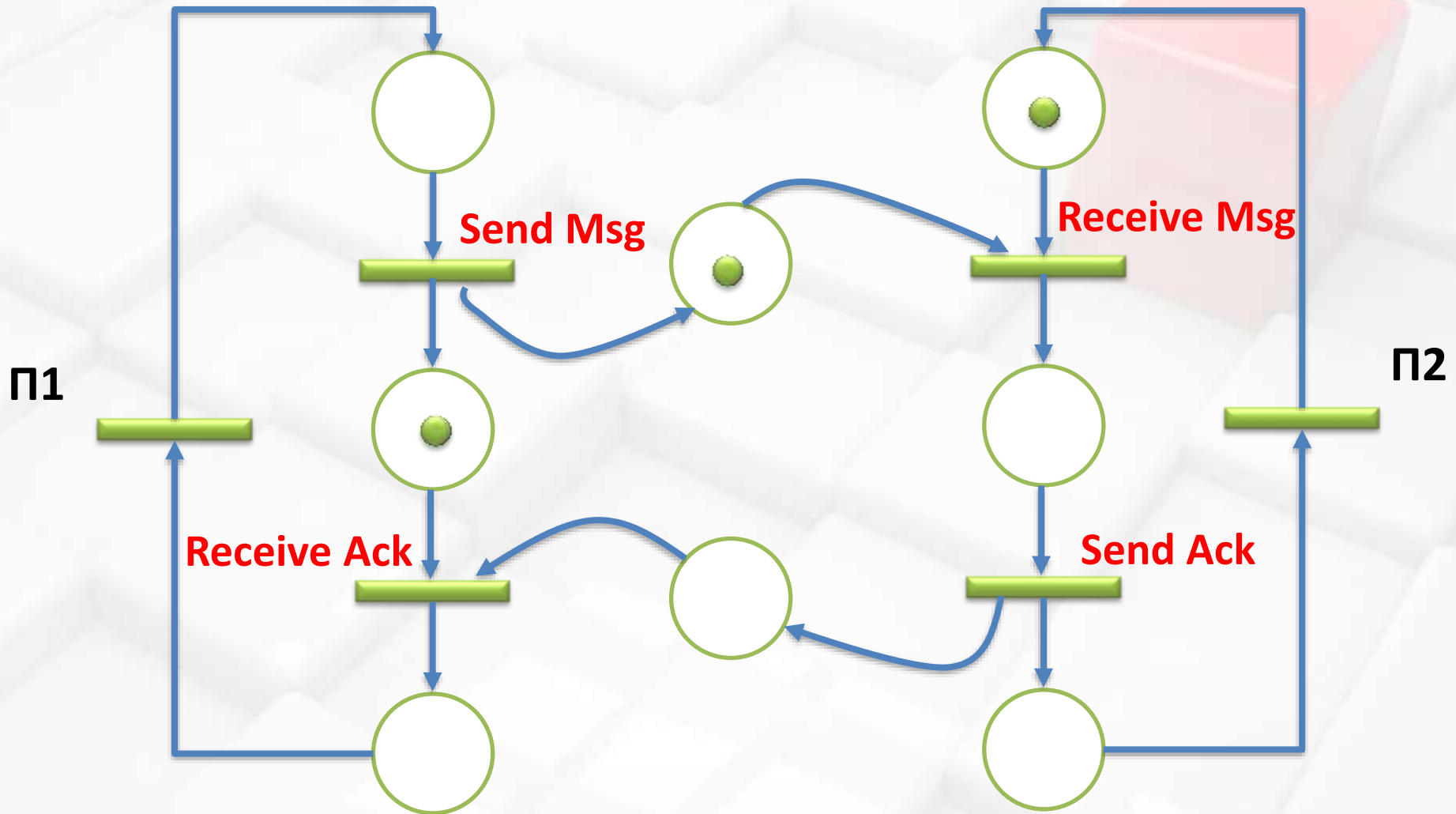
Модел на критична секция



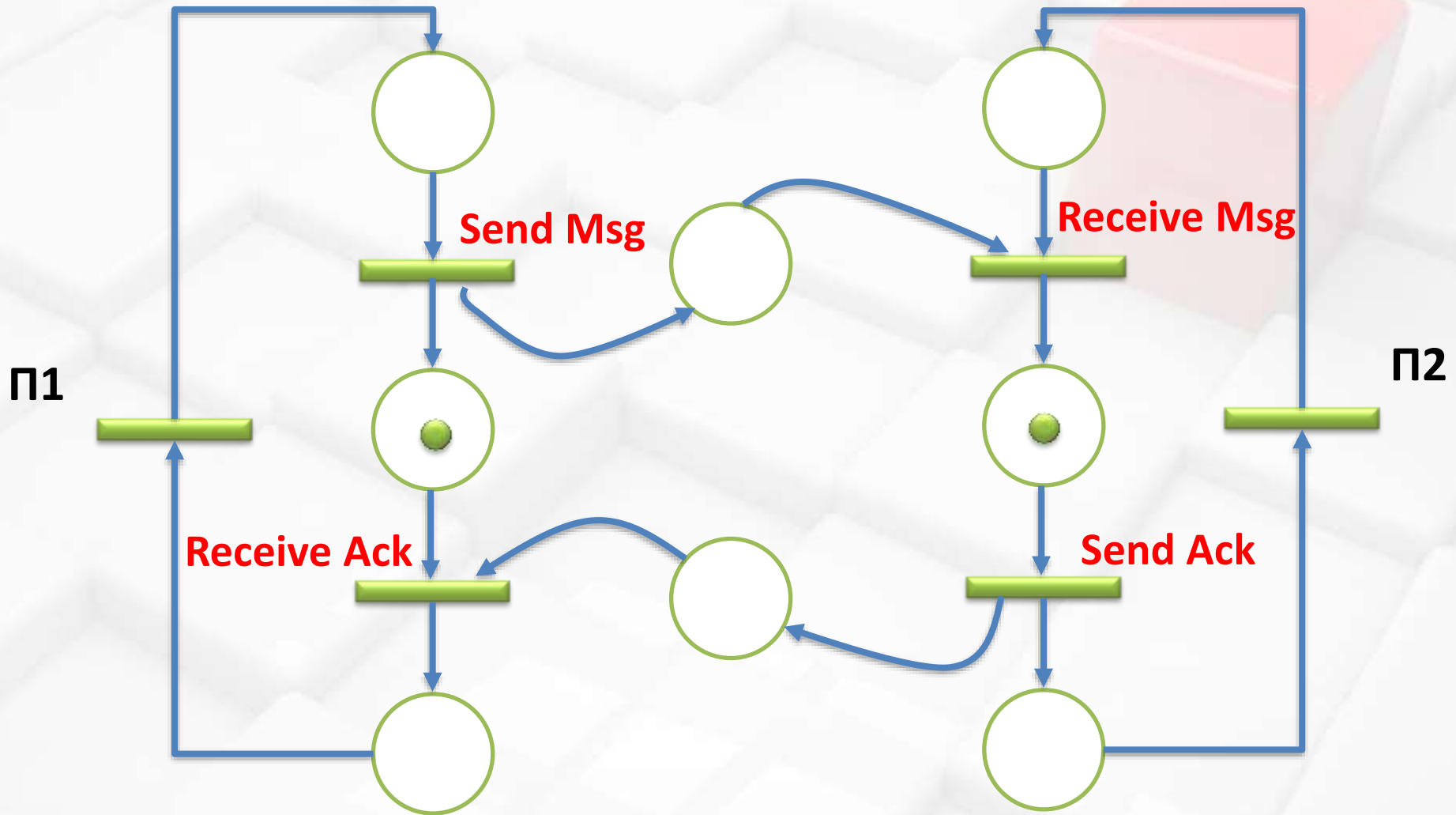
Комуникационен протокол



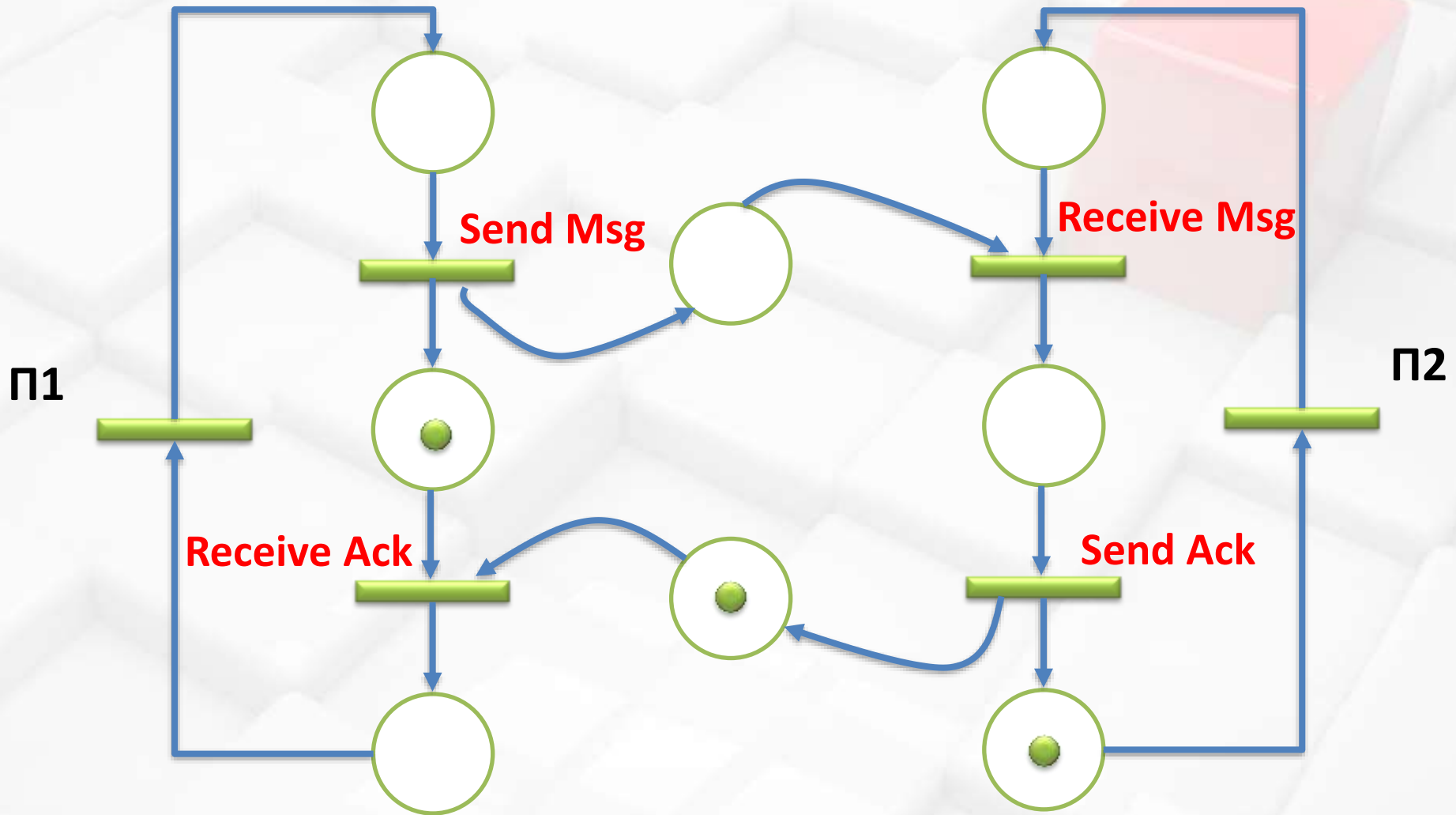
Комуникационен протокол



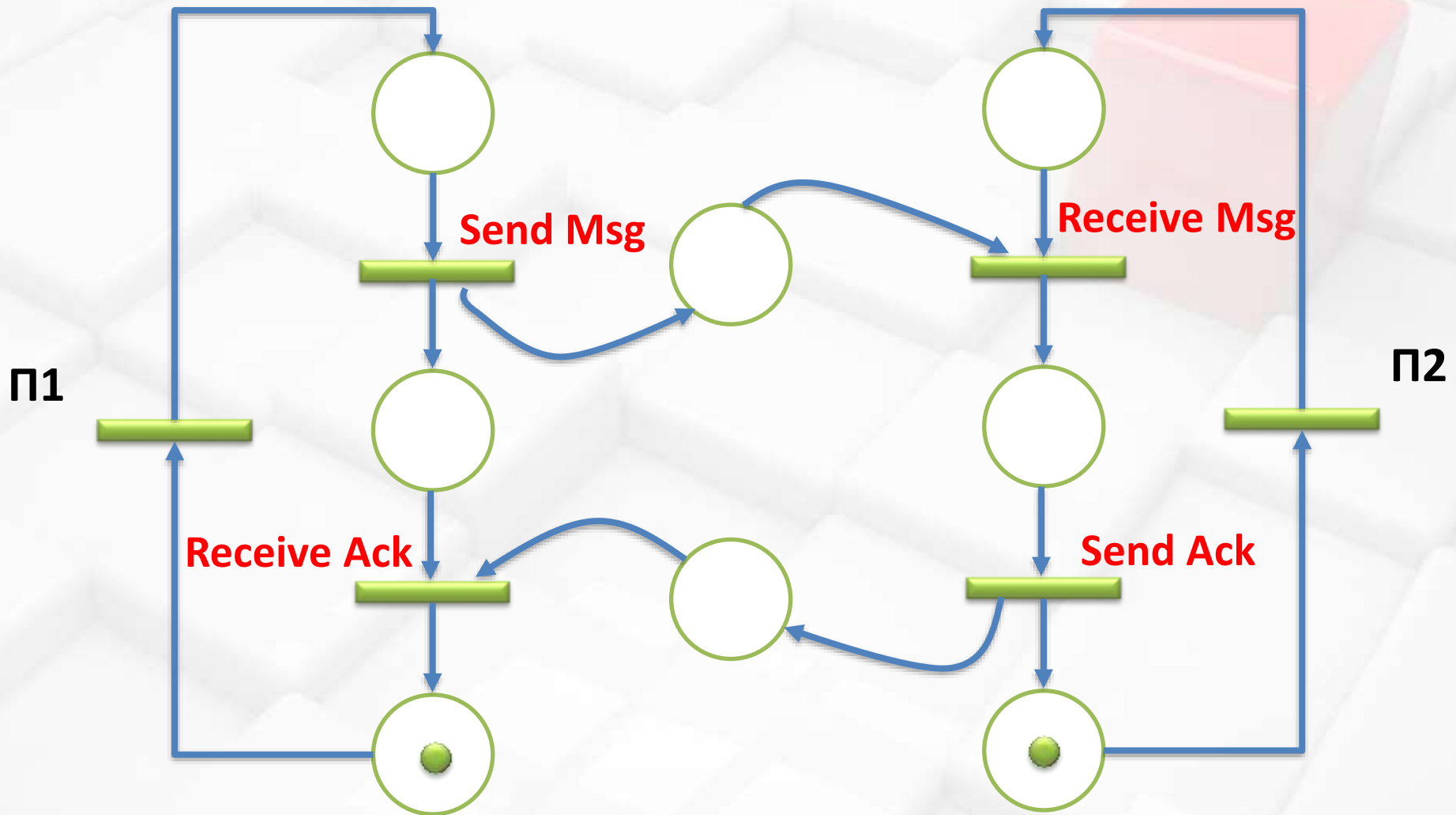
Комуникационен протокол



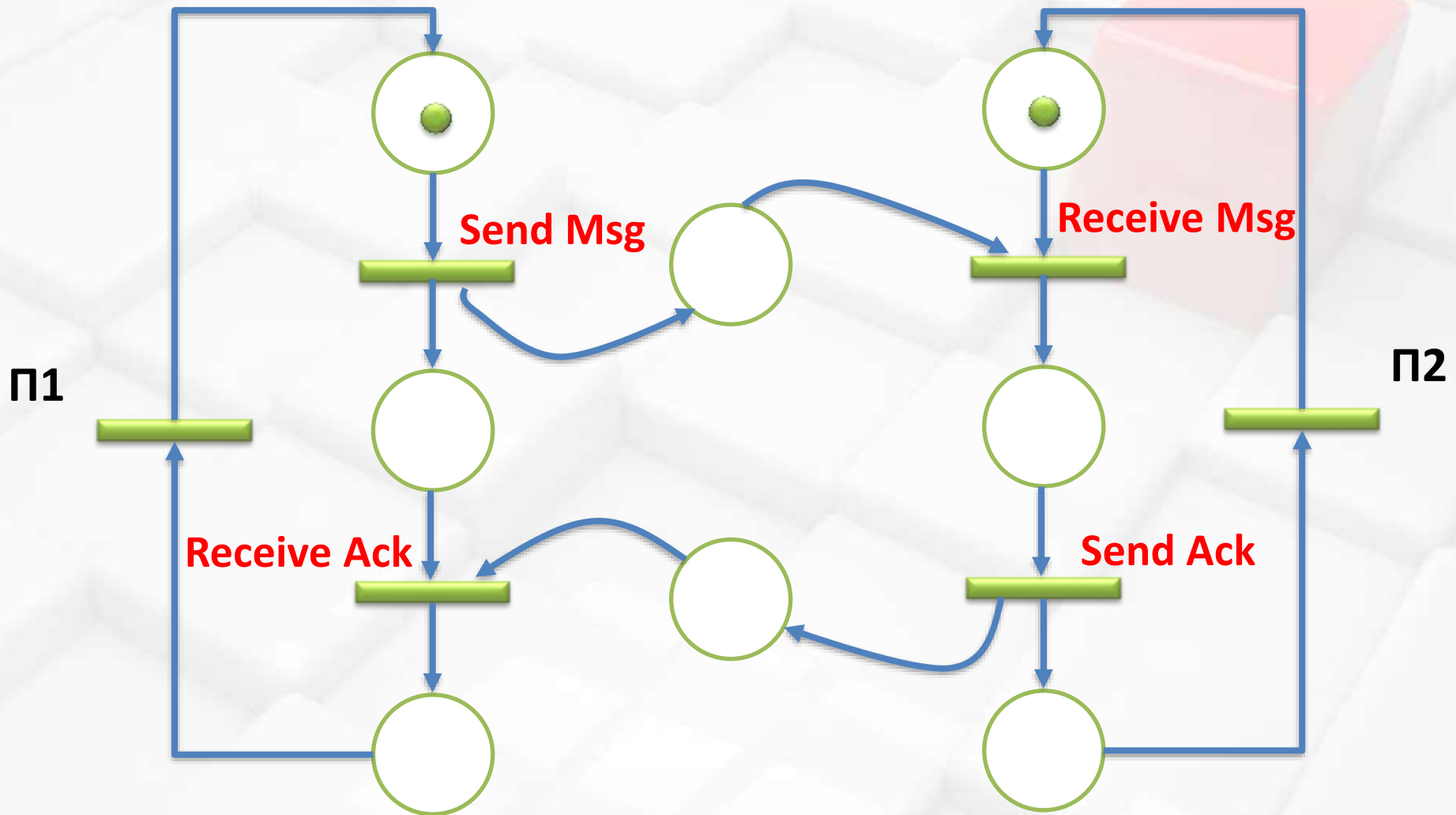
Комуникационен протокол



Комуникационен протокол

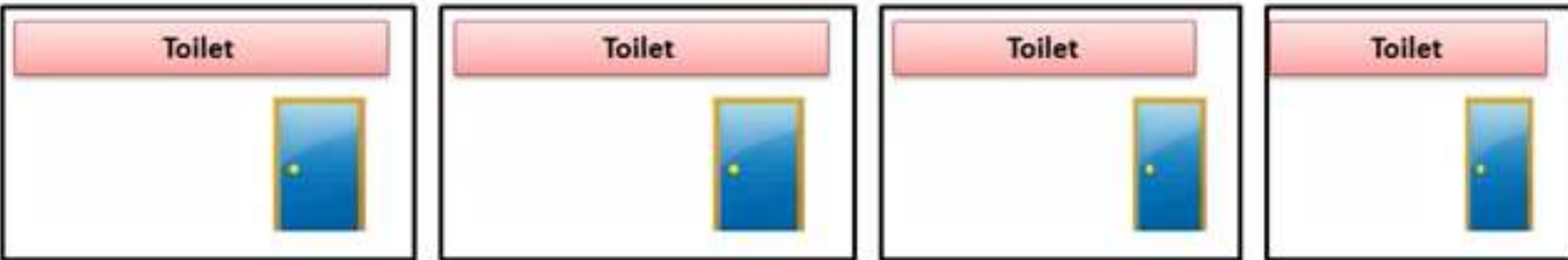


Комуникационен протокол



**Използване на семафори за
едновременен достъп на
процес до много ресурси**

Пример със семафор

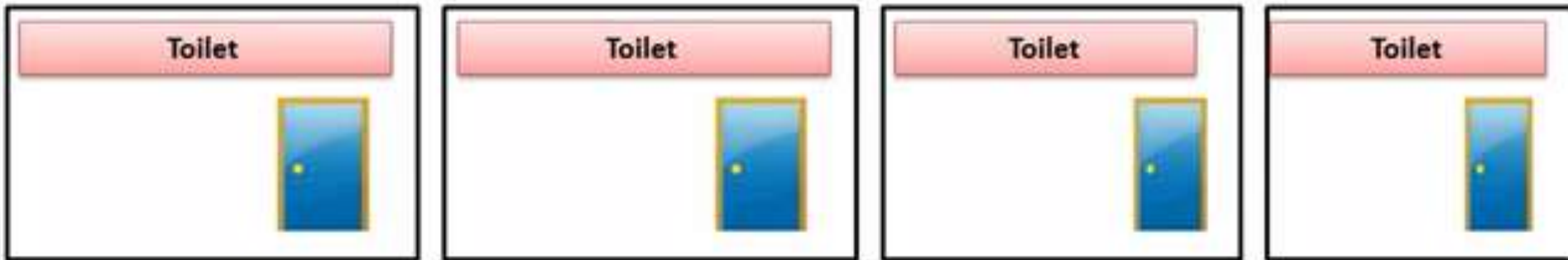


Четири идентични ключа

Ключове = 4



Пример със семафор

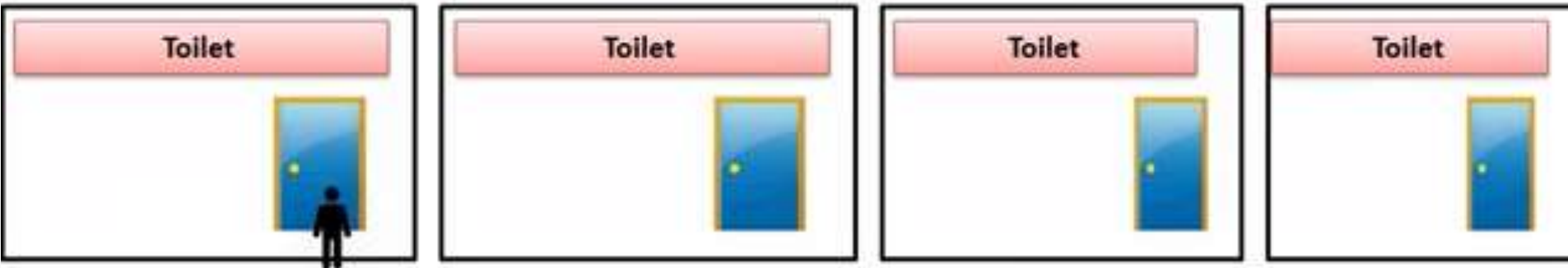


Четири идентични ключа

Ключове = 4



Пример със семафор

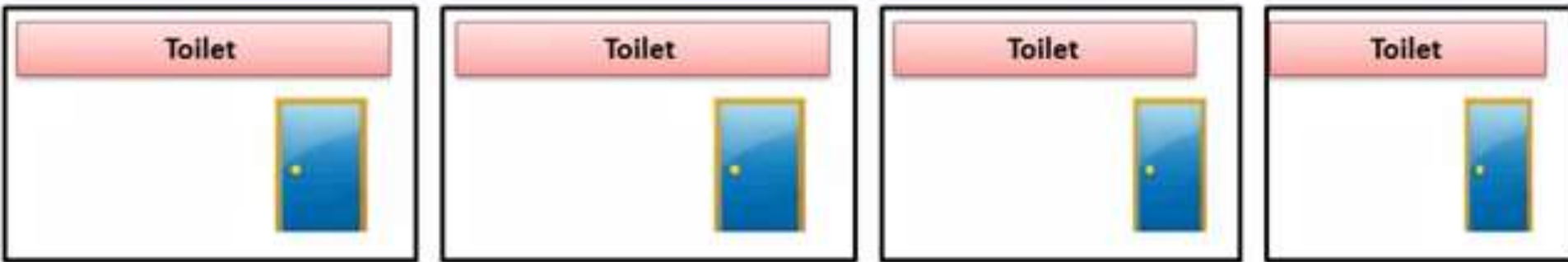


Четири идентични ключа

Ключове = 3



Пример със семафор

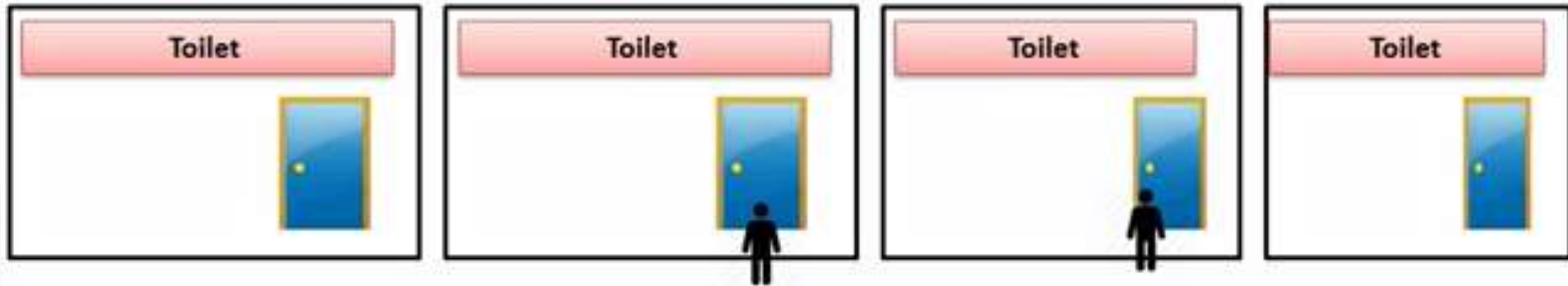


Четири идентични ключа

Ключове = 3



semaphore

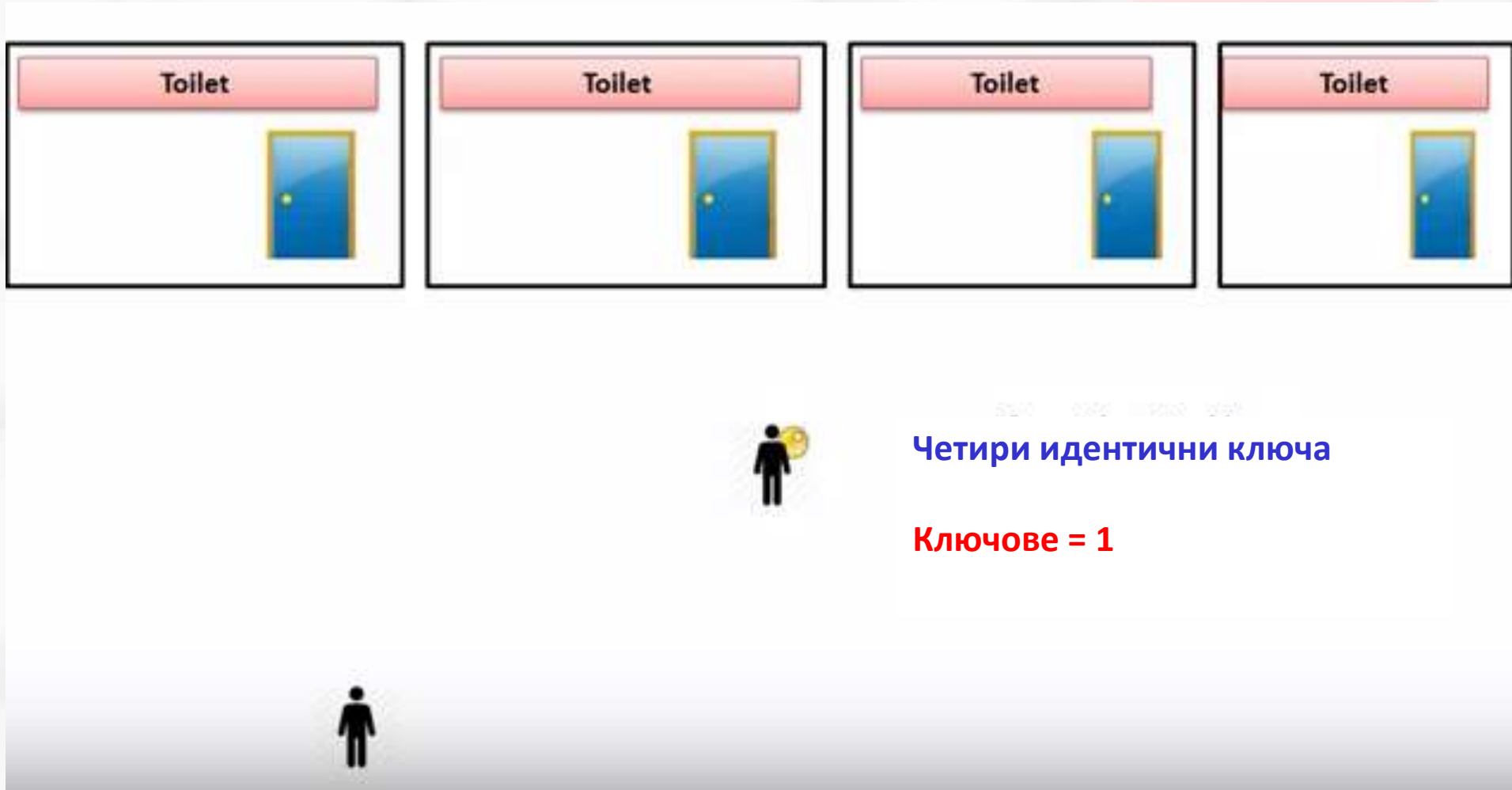


Четири идентични ключа

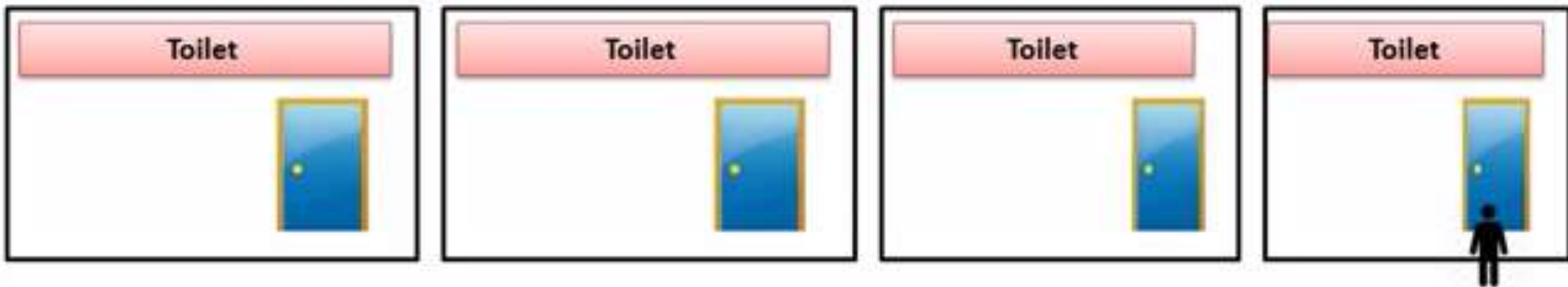
Ключове = 2



Пример със семафор



Пример със семафор

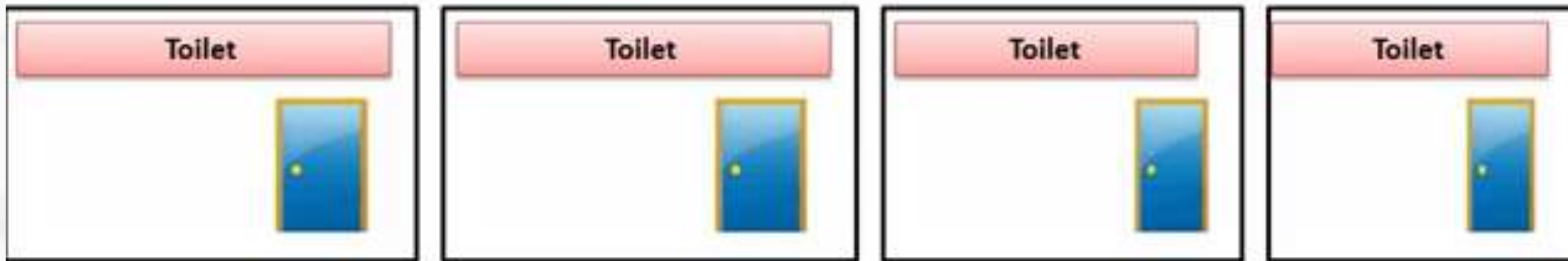


Четири идентични ключа

Ключове = 0



Пример със семафор



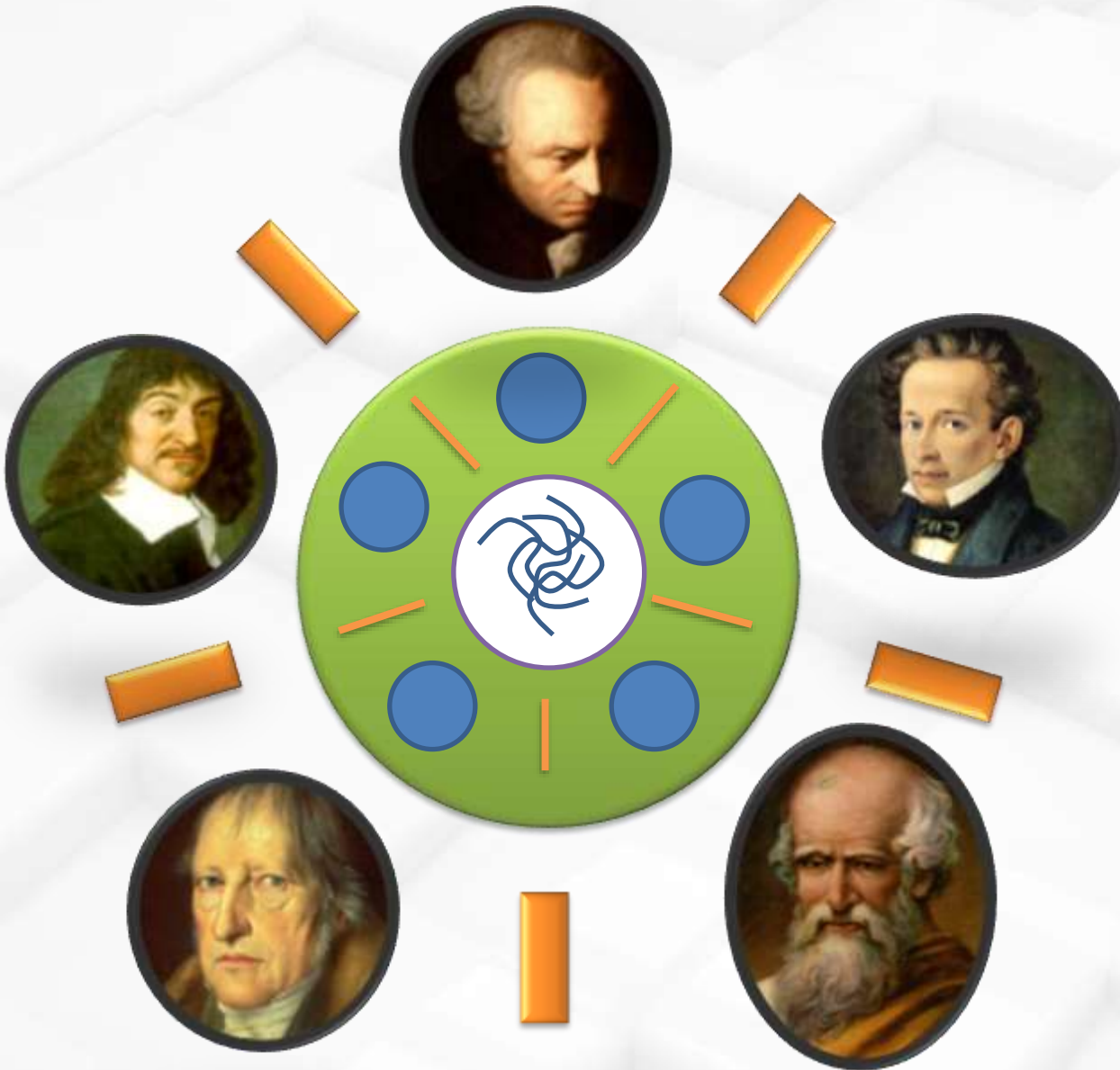
Няма ключове, значи
Ангел ще чака.



Четири идентични ключа

Ключове = 0

Вечерящите философи



Този пример се използва за създаване на **паралелни алгоритми** с цел илюстриране техниките за **синхронизация** и комуникацията между паралелни процеси.

Вечерящите философи

Пет тихи философи седят на кръгла маса, на която има чиния със спагети. Пет клечки са разположени между всяка двойка съседни философи.

Всеки философ трябва **последователно** да **мисли** и **яде**. Един философ може да яде спагети, само когато той има и двете: лява и дясна клечки. Всяка клечка може да се използва само от един философ и така, че даден философ може да използва клечката само ако не се използва от друг философ.

След като даден философ приключи яденето, той трябва да остави и двете клечки, така че те да станат достъпни за другите. Философите могат да вземат клечка отдясно или отляво, когато станат свободни, но не могат да започнат да се хранят преди и двете клечки да са в ръцете им.

Храненето не е ограничено по отношение на количеството спагети или стомашното пространство; безкрайна доставка и прием на спагети е гарантирана.

Проблемът е как да се разработи модел на поведение (**паралелен алгоритъм**), така че всеки философ може вечно да продължи да редува хранене и мислене, като се предполага, че никой от тях не знае кога другите философи искат да ядат или мислят.

Dining Philosophers Problem

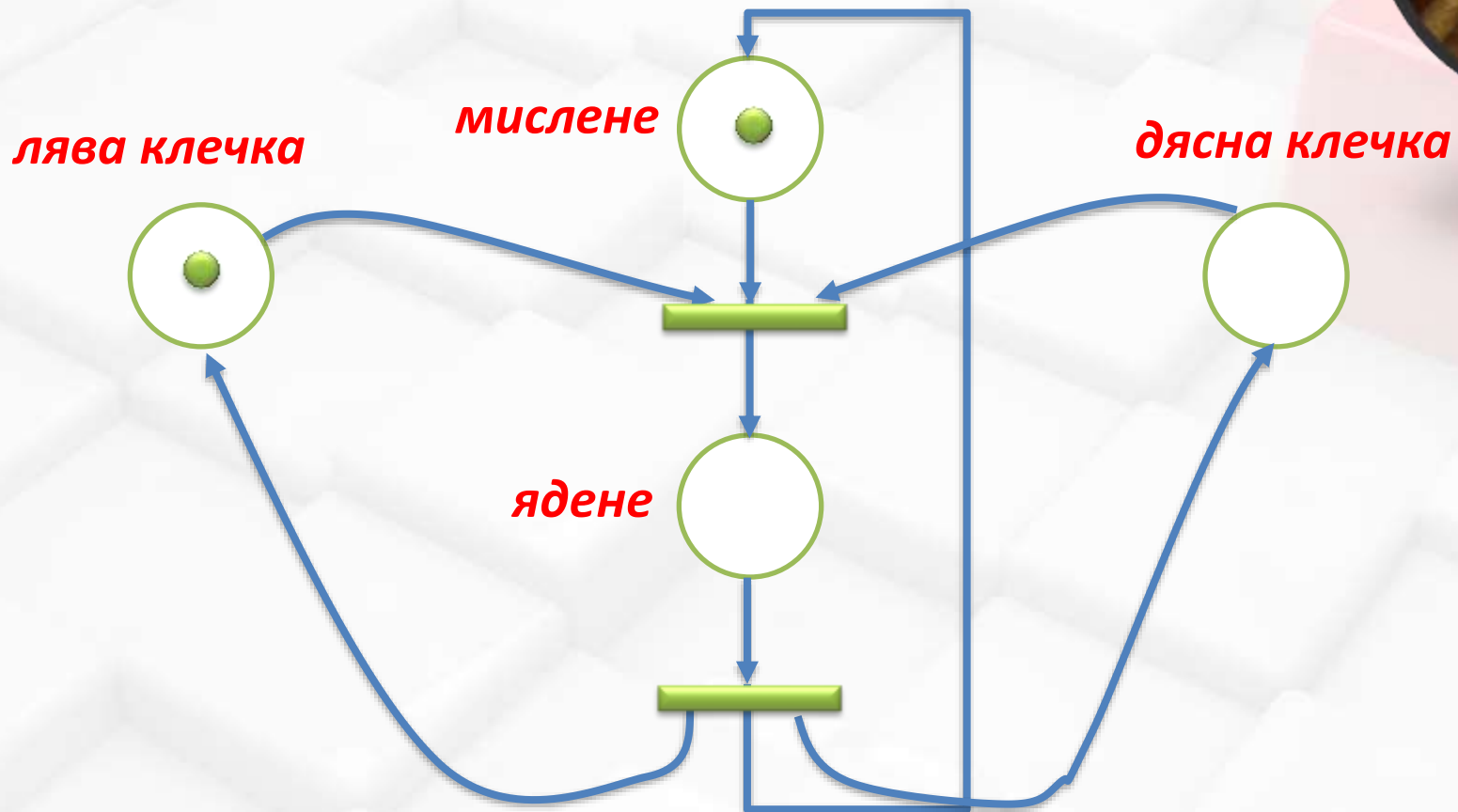


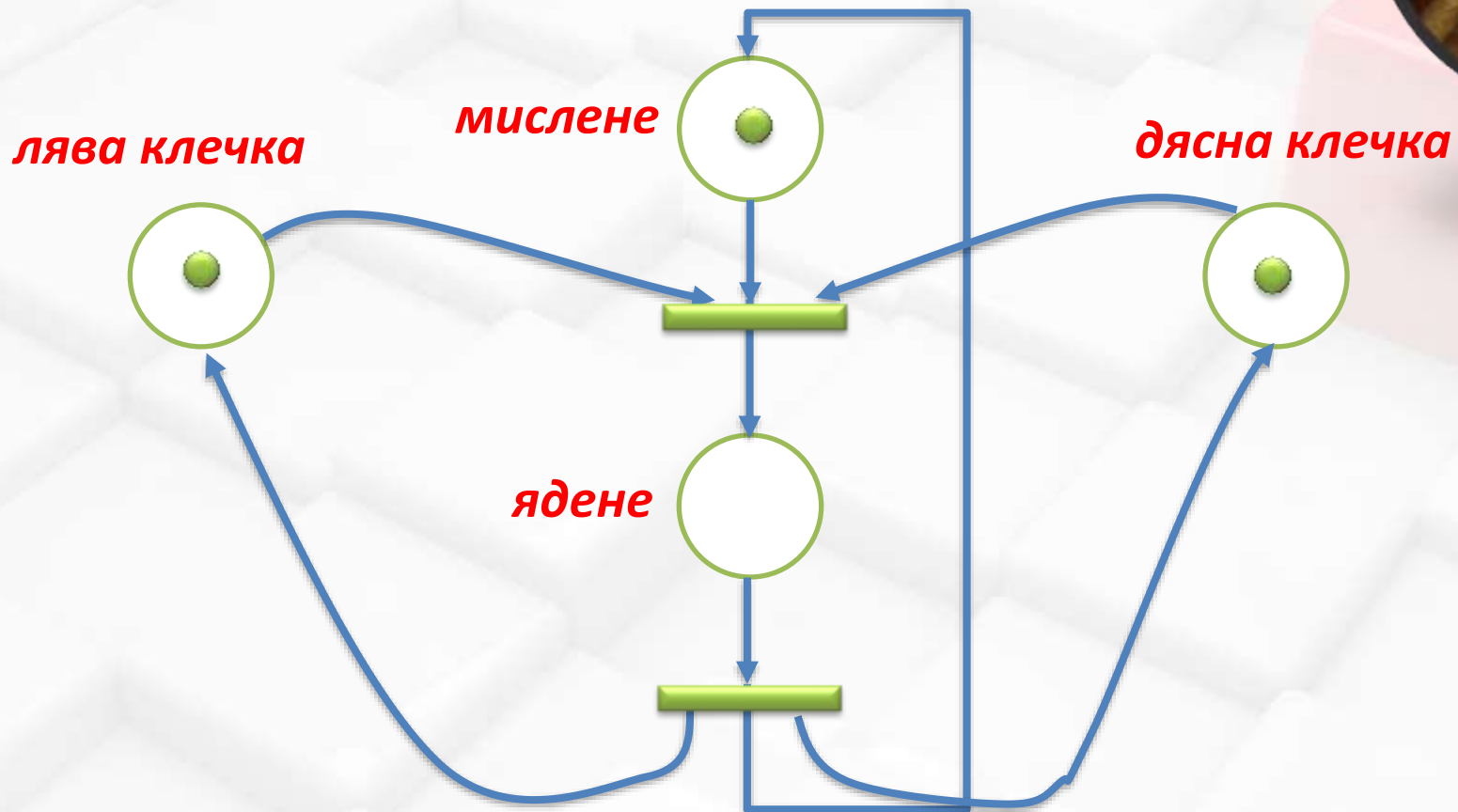
<https://www.youtube.com/watch?v=XjIFoND00oY>

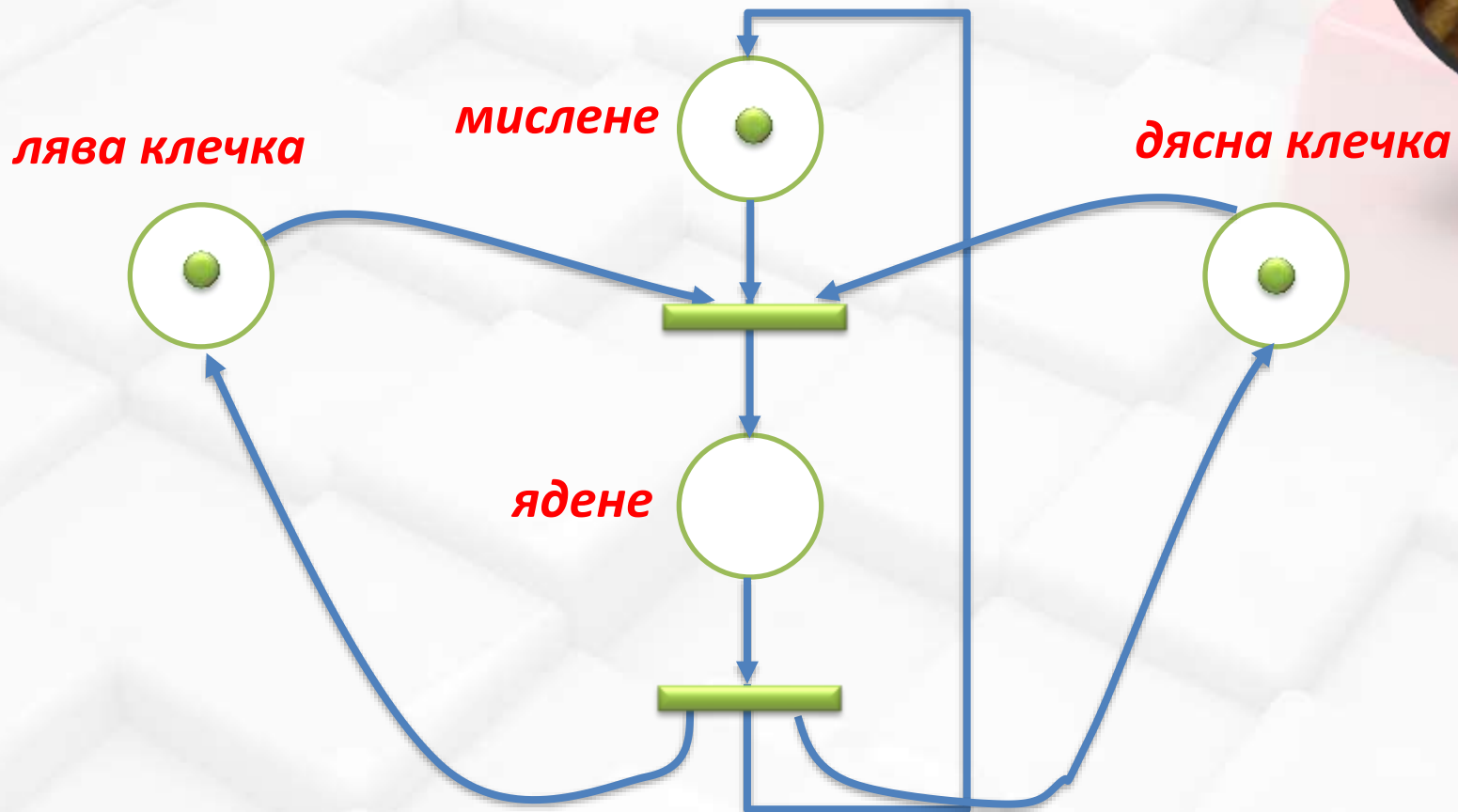
Да моделираме този проблем с Мрежи на Петри.

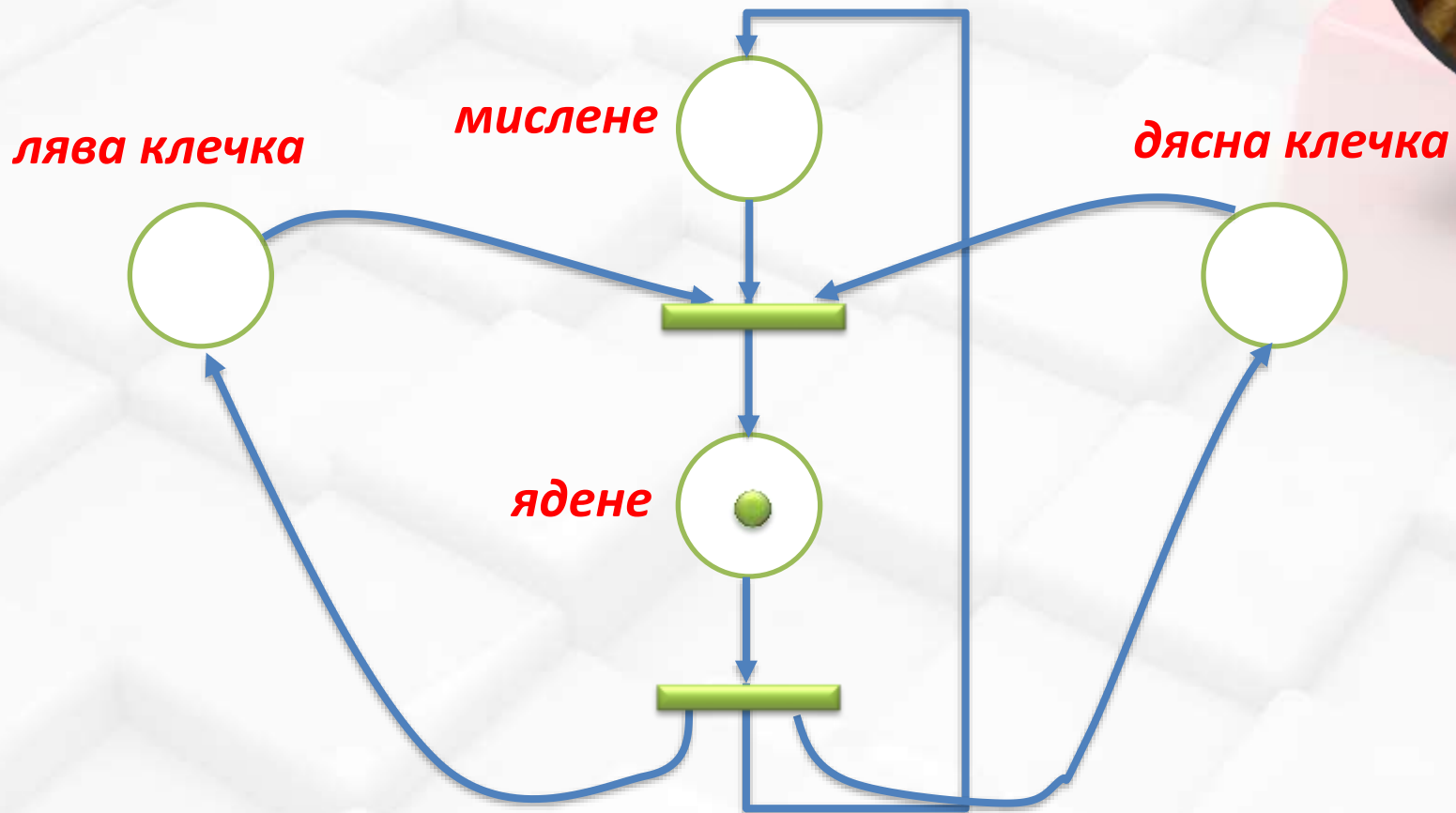
Deadlock – два съседни философи не могат да седат едновременно на масата

Гладуване – ако двама от философите са по-бързи от другите, то другите трима ще бъдат в режим на чакане непрекъснато









```
do
{
  P(chopstick[i]);
  P(chopstick[(i+1)%5]);

  eat;

  V(chopstick[i]);
  V(chopstick[(i+1)%5]);

  think;
} while (true)
```



2 проблема:
Deadlock
Гладуване

Друг вариант



Реализиран с кръгов буфер размер N=3 (свободни)

Друг вариант



Реализиран с кръгов буфер размер $N=3$ (свободни)

Друг вариант



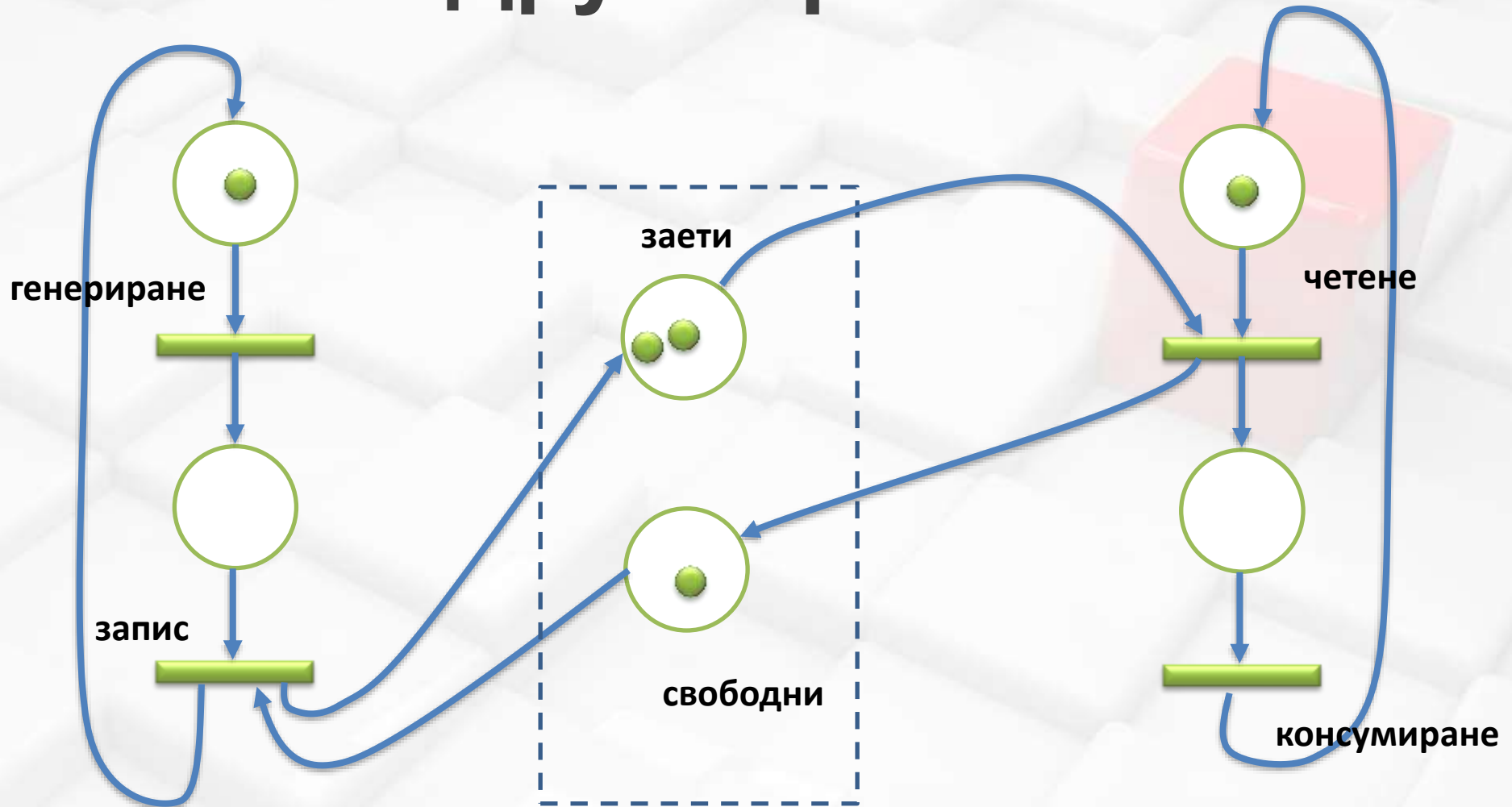
Реализиран с кръгов буфер **запис** $N=3$ (2 свободни 1 заети)

Друг вариант



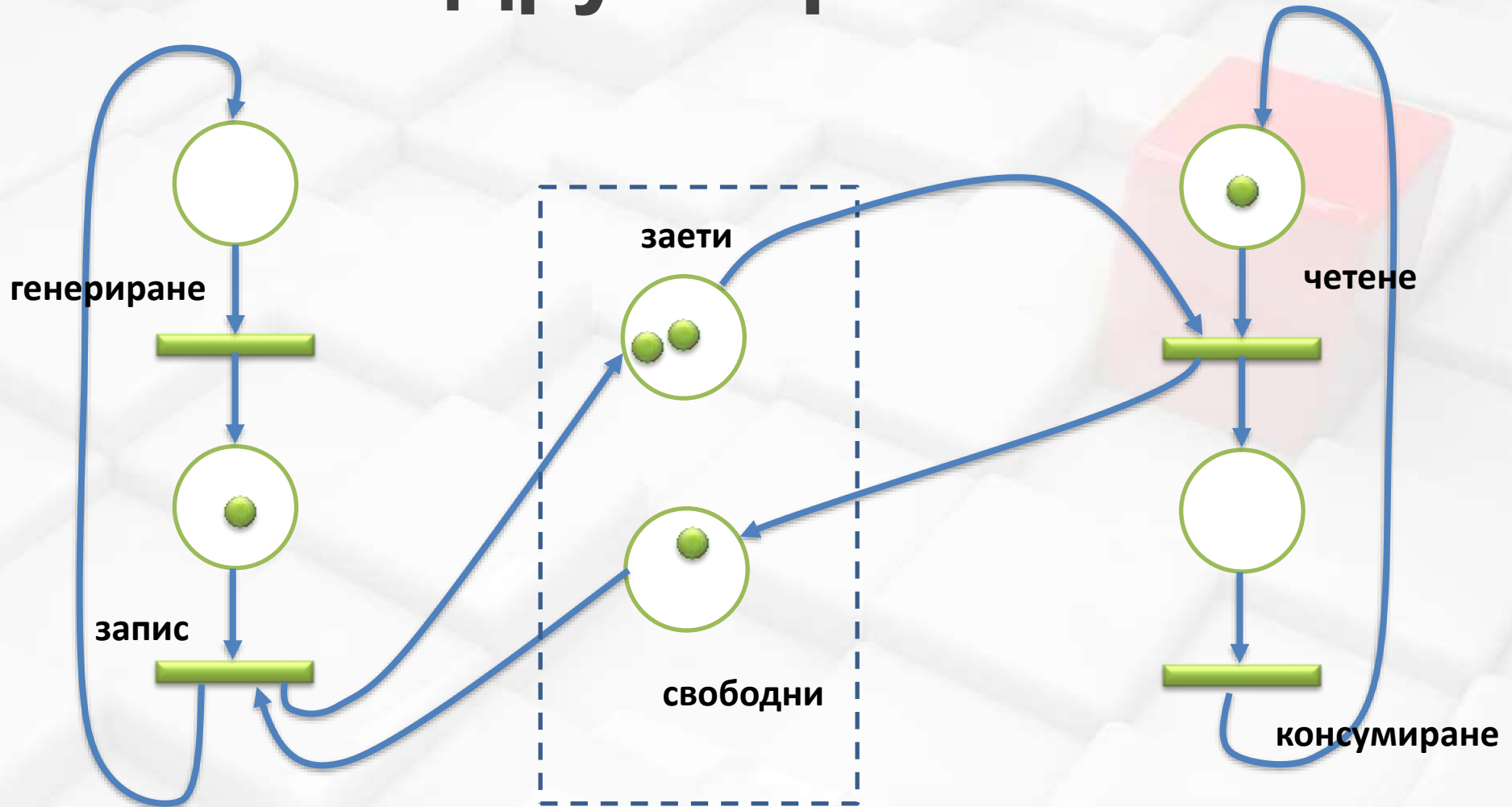
Реализиран с кръгов буфер **генериране** $N=3$ (2 свободни 1 заети)

Друг вариант



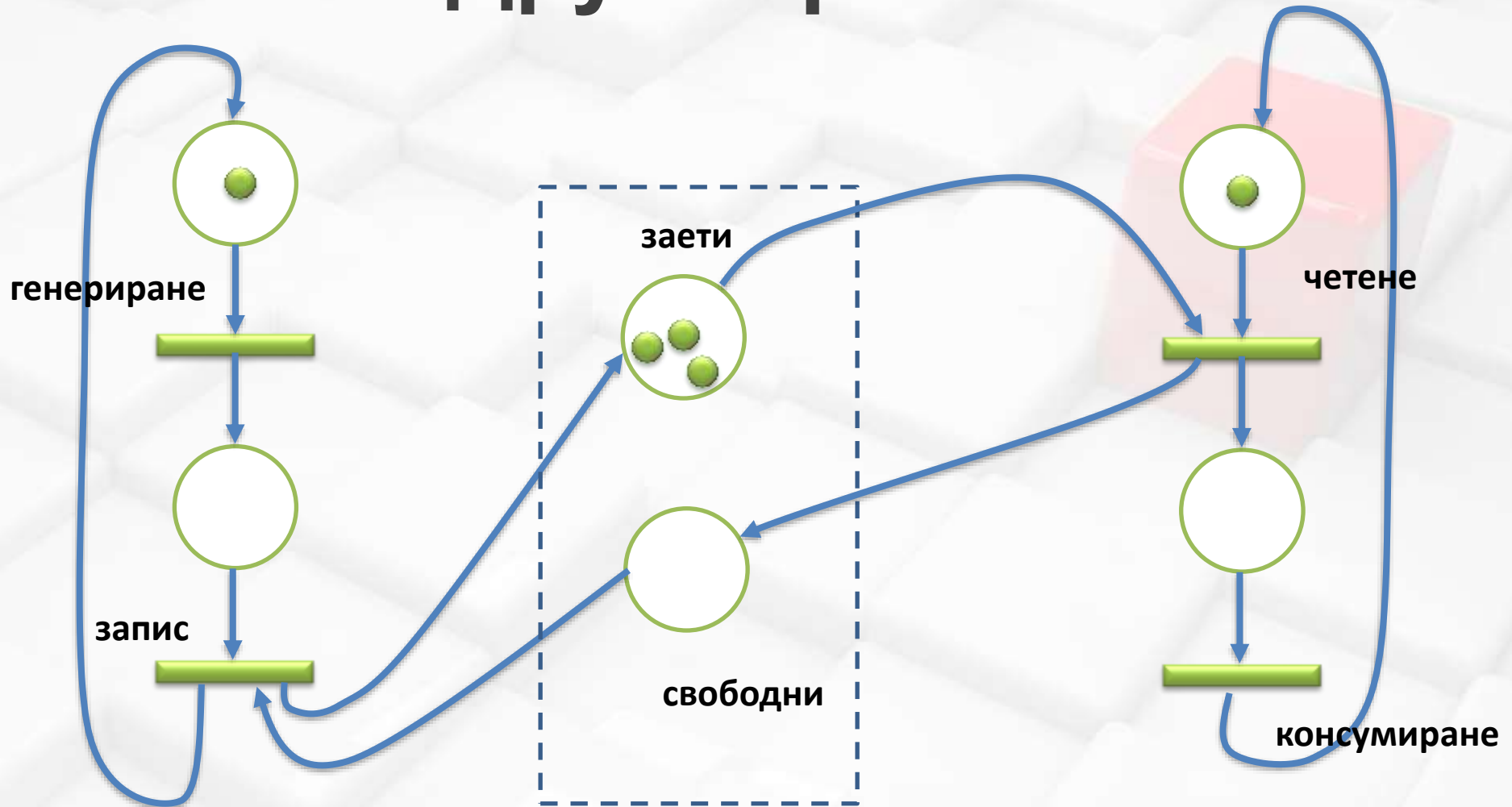
Реализиран с кръгов буфер **запис** $N=3$ (1 свободни 2 заети)

Друг вариант



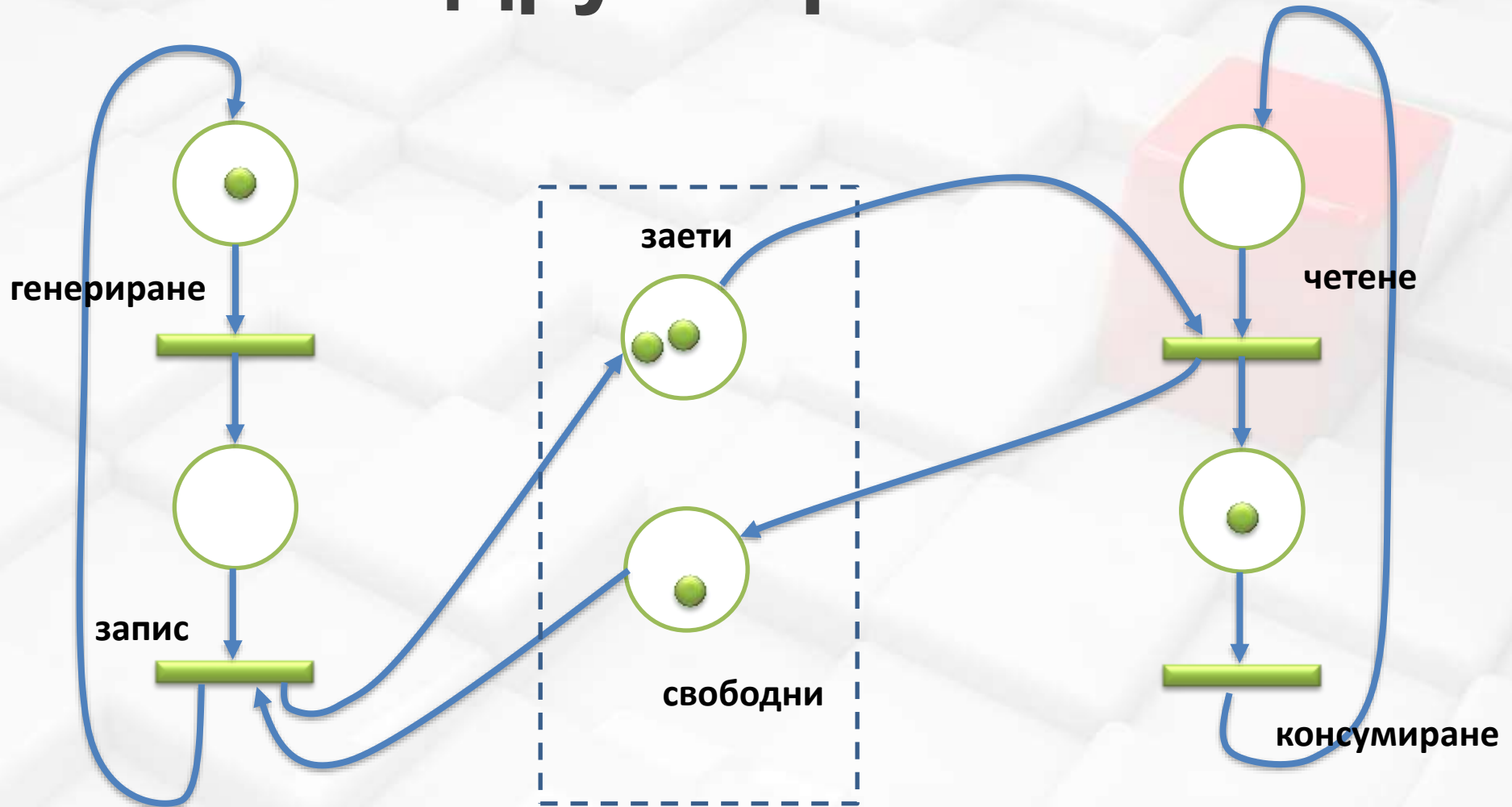
Реализиран с кръгов буфер **генериране** $N=3$ (1 свободни 2 заети)

Друг вариант



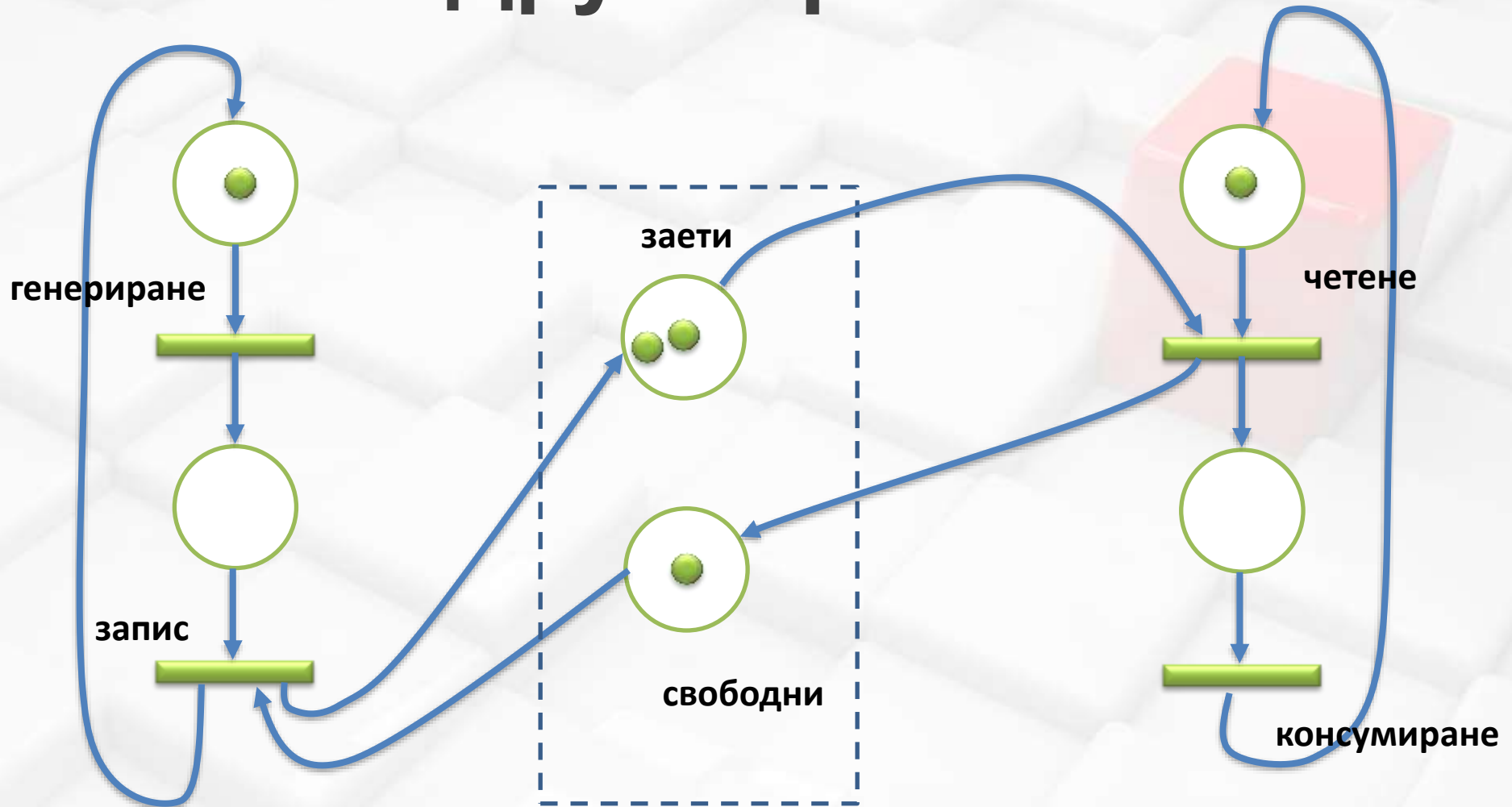
Реализиран с кръгов буфер **запис** $N=3$ (0 свободни 3 заети)

Друг вариант



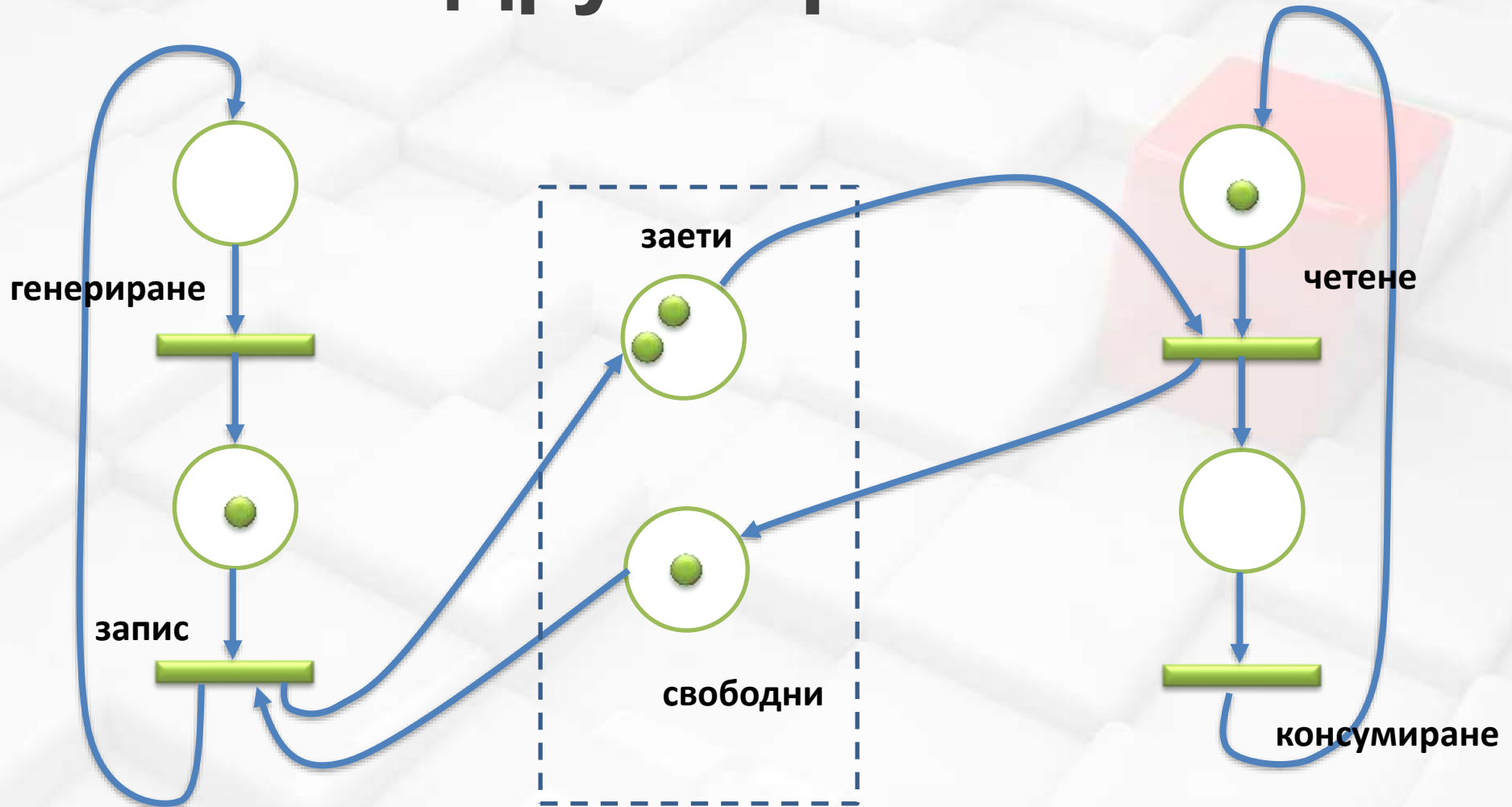
Реализиран с кръгов буфер **четене** N=3 (1 свободни 2 заети)

Друг вариант



Реализиран с кръгов буфер **консумиране** $N=3$ (1 свободни 2 заети)

Друг вариант



Реализиран с кръгов буфер **генериране** $N=3$ (1 свободни 2 заети)

Друг вариант



Реализиран с кръгов буфер **четене** $N=3$ (2 свободни 1 заети)

Друг вариант



Реализиран с кръгов буфер **запис+консумиране** $N=3$ (1 свободни 2 заети)

<https://www.youtube.com/watch?v=GvfjiA9jkTs>

